M05 — Consolidación de Blazor y despliegue elemental

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: TEORÍA

AUTOR: JAIME GALLO

Índice del módulo

- 5.1 — Consolidación del proyecto Blazor
- 5.2 — Ejecutar, compilar y publicar ASP.NET Core
- 5.3 — Kestrel y la aplicación publicada
- 5.4 — Hospedaje elemental con IIS
- 5.5 — Docker: imagen, contenedor y puertos

Presentación del módulo

Una aplicación web no está terminada porque sus archivos compilen. Debemos entender cómo cooperan los componentes de interfaz, los servicios y el servidor, y distinguir el proceso de desarrollo del proceso que ejecutará una publicación. Este último módulo reúne dos recorridos que se apoyan en conocimientos ya adquiridos: el cierre funcional del cliente **Blazor B** y la publicación elemental de la aplicación **MVC A**.

El curso continúa utilizando .NET 8, Visual Studio y, cuando corresponde al CRUD, SQL Server Express LocalDB. La base no cambia a SQLite para simplificar un despliegue. Tampoco se presenta el laboratorio como guía de operación empresarial: la configuración de HTTPS público, proxies externos, endurecimiento, contenedores con persistencia y automatización de infraestructura quedan fuera del itinerario.

En el punto 5.1 observaremos un recorrido Blazor → HTTP → API → respuesta → componente. En 5.2–5.5 compararemos la misma aplicación ASP.NET Core MVC bajo tres formas elementales de ejecución: publicación con Kestrel, hospedaje mediante IIS y contenedor Docker. Los ejemplos explicativos de este manual pueden ser independientes; la secuencia de construcción exacta, sus rutas y sus estados pertenecen al manual de prácticas.

La secuencia presencial reserva 60 minutos para conceptos, 270 minutos para construcción y verificación, y 30 minutos para dudas. Los estudios de ampliación no añaden obligaciones a esa jornada.

5.1. Consolidación: seguir una operación Blazor desde el navegador hasta la API

5.1.1. El sistema es más que la suma de sus archivos

Al terminar M03 la aplicación Blazor representaba información y actualizaba estado mediante componentes interactivos. Durante M04 se incorporó una API HTTP pequeña y se trasladó la comunicación a un servicio tipado. Ahora debemos explicar el comportamiento del conjunto sin perder de vista una separación fundamental: **una lista dibujada en el navegador, una colección que vive en un componente y los datos que conserva la API no son el mismo objeto ni tienen necesariamente la misma duración**.

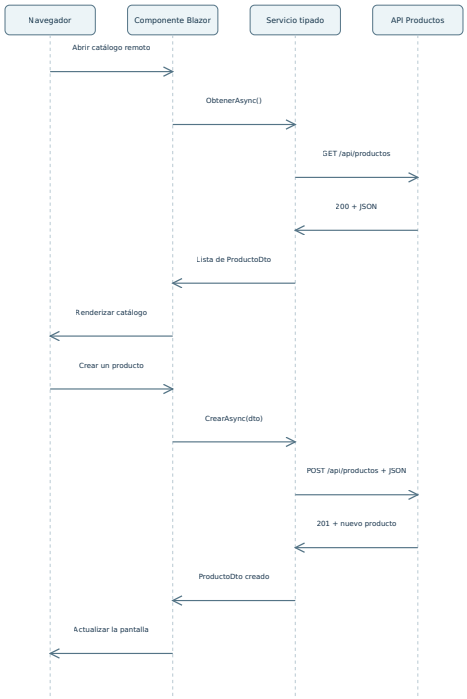
Pensemos en una tienda de material de oficina. Cuando alguien abre «Catálogo remoto», no es la página HTML la que consulta una base SQL Server. El host Blazor Interactive Server crea o utiliza un circuito interactivo, construye el componente, resuelve el servicio configurado y este hace GET a la API. La API devuelve JSON que el cliente transforma en objetos .NET; finalmente, Blazor renderiza el estado. Si la API deja de responder, no desaparece necesariamente el servidor Blazor: la interfaz sigue siendo capaz de mostrar un error que explica el fallo.

Conviene separar seis responsabilidades:

Pieza	Qué conoce	Qué no debería decidir
Componente Razor	Estado de pantalla, presentación y eventos	Direcciones HTTP repetidas y contratos de transporte
DTO de producto	Campos intercambiados con la API	Ciclo de vida visual o acceso a una base
Interfaz de servicio	Operaciones de consulta y creación	Componentes concretos que las utilizan
Servicio HTTP	URL relativa, serialización y errores	Cómo se diseña la pantalla
HttpClient	Transporte, cabeceras, conexión y timeout	Reglas de negocio de productos
API	Validación del dato, identificador y respuesta HTTP	Vista que el cliente mostrará

El beneficio no es puramente estético. Si otra interfaz consume la API, el endpoint puede mantenerse. Si la API cambia de dirección, el cliente tipado puede volver a configurarse sin modificar la presentación. Si un producto es inválido, corresponde al servidor rechazarlo aunque la pantalla hubiera realizado comprobaciones previas.

5.1.2. Diagrama de comunicación y modelo Interactive Server



El diagrama representa una llamada HTTP desde **el servidor Blazor** en Interactive Server, no un fetch automático iniciado por JavaScript en el navegador. Esta diferencia afecta a las reglas CORS y al diagnóstico de conectividad: en este recorrido quien necesita llegar a la API es el proceso ASP.NET Core que ejecuta Blazor. La web puede abrirse correctamente mientras falla la consulta remota.

Recibir 200 no equivale a disponer de un DTO válido: el cuerpo podría no ser JSON. Por eso existe una etapa de deserialización y después otra de actualización del estado, ambas susceptibles de error. La operación POST añade un identificador que decide el servidor. Si dos personas crean productos al mismo tiempo, la interfaz no debe adivinar el próximo ID a partir del número de elementos visibles.

5.1.3. Ejemplo teórico independiente: clasificar fallos HTTP

Antes de estudiar el proyecto de prácticas, consideremos un servicio de lectura de avisos. Permite distinguir una respuesta HTTP de error, un fallo de conexión y una respuesta imposible de deserializar. El ejemplo presupone un proyecto ASP.NET Core con los tipos definidos y el espacio de nombres `System.Net.Http.Json`.

```
using System.Net.Http.Json;

public sealed record AvisoDto(int Id, string Texto);

public sealed class AvisosHttpService(HttpClient http)
{
    public async Task<IReadOnlyList<AvisoDto>> ObtenerAsync(
        CancellationToken cancelacion = default)
    {
        using var respuesta =
            await http.GetAsync("api/avisos", cancelacion);

        // Una respuesta recibida también puede ser 404 o 500.
```

```
        respuesta.EnsureSuccessStatusCode();

        // HTTP 200 no garantiza por sí solo JSON válido.
        return await respuesta.Content
            .ReadFromJsonAsync<List<AvisoDto>>(cancelacion)
            ?? throw new InvalidOperationException(
                "Se esperaba una colección de avisos.");
    }
}
```

La API de avisos es solo un escenario explicativo: **no constituye una tercera aplicación que el alumno tenga que construir**. Para usar este servicio sería necesario registrar un HttpClient con dirección base y proporcionar un endpoint compatible.

Si el servidor devuelve 404, `EnsureSuccessStatusCode` genera `HttpRequestException`. Si no hay conexión, también puede producirse `HttpRequestException`, pero sin una respuesta 404 que examinar: no debe confundirse ausencia de respuesta con «recurso inexistente». Si se supera el timeout, puede manifestarse como `TaskCanceledException`. Una respuesta 200 con JSON incompatible puede producir `JsonException`. Los mensajes del componente deben corresponder a la causa observada, evitando atribuir indiscriminadamente todos los fallos a una misma capa.

5.1.4. GET, POST y representación de datos

GET solicita la representación actual de un recurso. POST solicita en este curso crear uno nuevo; una operación satisfactoria devuelve **201 Created** con representación del nuevo producto y una ubicación. Por ejemplo:

```
HTTP/1.1 201 Created
Location: /api/productos/3
Content-Type: application/json

{"id":3,"nombre":"Regla","precio":1.75}
```

Un cliente puede añadir el DTO devuelto a la lista que ya representa o consultar de nuevo el catálogo. La primera opción evita otra petición, pero esa lista podría estar desactualizada si existen otros clientes. La segunda opción obtiene otra representación del servidor, con coste de red y con una nueva posibilidad de error.

Ninguna opción cambia el modelo de persistencia. La API docente de M04 aloja una colección de productos **en memoria**. Reiniciar su proceso restaura los productos iniciales. Por el contrario, el CRUD MVC de M02 usa SQL Server Express LocalDB: es otro modelo de almacenamiento y otra competencia. Una recarga del componente, un reinicio del servidor Blazor y un reinicio de la API son acciones distintas, con efectos distintos.

5.1.5. Validación local y validación del servidor

Una comprobación local evita envíos innecesarios y permite corregir pronto los datos:

```
public sealed record CrearProductoDto(string Nombre, decimal Precio);

public static class ReglasEntrada
{
    public static bool EsValida(CrearProductoDto entrada) =>
        !string.IsNullOrWhiteSpace(entrada.Nombre) &&
```

```
        entrada.Precio > 0;
}
```

No obstante, el servidor debe comprobar el contrato porque otros clientes pueden llamarlo directamente. La interfaz no es una barrera de integridad. En el laboratorio:

Resultado	Significado	Acción pedagógica
200 OK en GET	Se obtuvo una representación	Comprobar JSON y renderizado
201 Created en POST	El recurso fue creado	Consultar Location y nuevo ID
400 Bad Request	La entrada no satisface el contrato	Corregir nombre/precio
404 Not Found	No existe la ruta/recurso solicitado	Revisar ruta e identificador
Error de red	No se obtuvo respuesta HTTP utilizable	Comprobar proceso y puerto

Esta distinción permite diagnosticar de forma causal: que el botón no funcione no demuestra que el problema esté en Blazor. Debemos observar la petición y la respuesta en el punto adecuado.

5.1.6. El servicio tipado e inyección de dependencias

El registro de un cliente tipado asocia `IProductosApiService` con `ProductosApiService` y proporciona HttpClient ya configurado. El componente consume la interfaz, no una URL repetida:

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri("http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

El fragmento presupone que ambos tipos existen. La dirección base de ejemplo es local; puede leerse desde `appsettings.json` para evitar cambiar código al usar otro puerto. Las rutas relativas de los métodos del servicio deben combinarse con esa dirección. No se modifica la clase de presentación simplemente para corregir la ubicación del servidor.

5.1.7. Seguridad: recordar límites, no bloquear el laboratorio

La ampliación Identity/JWT de M04 permanece independiente. El catálogo básico de M05 usa una API pública de productos en memoria; no necesita token JWT ni conectarse a las tablas de usuarios. Tampoco una interfaz se vuelve segura por ocultar un botón: la autorización es una decisión del servidor.

Este límite es fundamental para no volver irrealizable una jornada de seis horas. P5.1 necesita **Blazor B y API de productos**. La ampliación Identity/JWT puede revisitarse posteriormente, pero no se añade como dependencia del recorrido principal.

5.1.8. Diagnóstico causal antes de modificar código

Si Blazor abre pero no muestra la lista, abrir la URL de la API directamente permite comprobar si hay respuesta. Si devuelve 404, revisar ruta. Si devuelve 200 con contenido inesperado, revisar contrato JSON. Si funciona directamente pero falla desde Blazor, revisar `ProductosApi:BaseUrl`,

servicio registrado y errores del componente. Si el nuevo producto aparece y desaparece después del reinicio de la API, recordar que esta utiliza memoria.

La diferencia entre error de red, error HTTP, error de serialización, error de validación y estado local obsoleto debe poder explicarse antes de finalizar este punto. El alumno no necesita crear más arquitectura para demostrarlo; necesita poder seguir un recorrido ya construido y entender cada decisión.

5.1.9. Síntesis

El resultado formativo es una aplicación Blazor que consulta, crea, valida, informa de fallos y permite refrescar sus datos. Más importante todavía: el alumno es capaz de identificar quién mantiene cada estado y cuál es el origen de una respuesta. Esa comprensión permite pasar de «la pantalla funciona» a «sé por qué funciona y cómo repararla».

5.1.10. Una respuesta útil no es lo mismo que una pantalla actualizada

Conviene estudiar un problema que aparece incluso cuando ninguna línea de código contiene un error sintáctico. Imaginemos que la API conserva dos productos y que una persona abre el catálogo Blazor. El servidor Blazor obtiene dos DTO, los convierte en una lista y el componente dibuja dos elementos. Mientras esa persona sigue mirando la pantalla, otro cliente ejecuta un POST válido y la API pasa a conservar tres productos.

En ese instante existen dos afirmaciones compatibles: **el servidor dispone de tres productos** y **el componente representa dos**. La primera describe el estado del origen de datos; la segunda, una instantánea obtenida anteriormente. La aplicación no ha incorporado un mecanismo de notificación de cambios. Por tanto, no tiene por qué producirse una actualización espontánea en la interfaz.

Esta observación evita un diagnóstico equivocado. El sistema no ha perdido el tercer producto. Tampoco hay evidencia de que el renderizado esté roto. Para afirmar cualquiera de esas cosas habría que consultar al servidor y comparar ambas representaciones.

Modelo temporal de una consulta.

Instante	Acción	API	Lista del componente	Pantalla
T0	Arranca la API	Cuaderno, Lápiz	No creada	Sin catálogo
T1	Se abre Blazor y se completa GET	Cuaderno, Lápiz	Dos elementos	Dos elementos
T2	Un cliente externo crea Regla	Tres elementos	Dos elementos	Dos elementos
T3	El usuario solicita una nueva consulta	Tres elementos	Tres elementos	Tres elementos
T4	Se reinicia únicamente la API en memoria	Cuaderno, Lápiz	Tres elementos	Tres elementos
T5	Una nueva consulta termina con 200	Cuaderno, Lápiz	Dos elementos	Dos elementos

La tabla deliberadamente separa un cambio en el servidor de una modificación de la colección local. En T4 todavía pueden verse tres productos, aunque el nuevo proceso de API tenga dos. La interfaz solo se reconciliará con el servidor después de una consulta posterior. Si el GET de T5 falla, el resultado visual dependerá de cómo esté implementado el tratamiento de errores: se podría conservar la lista antigua o sustituirla por un estado de error. No hay que deducir la estrategia únicamente del código de la API.

Ejemplo conceptual de actualización explícita. Una función responsable de consultar no necesita conocer cómo se inició la operación; tanto el ciclo de vida como un botón pueden llamarla:

```
private async Task ActualizarAsync()
{
    var nuevos = await Servicio.ObtenerAsync();
    productos = nuevos.ToList();
}
```

Este fragmento es deliberadamente pequeño para mostrar **una única responsabilidad**: obtener una nueva representación y sustituir la anterior. No contiene manejo de errores, bandera de carga ni HTML. Todos esos aspectos se estudiaron por separado y se integran en el proyecto real. Si se añadieran directamente a este primer fragmento, sería más difícil ver que la sustitución de la colección es el punto exacto en que cambia el estado visible.

5.1.11. El ciclo de vida no sustituye los eventos de la interfaz

`OnInitializedAsync` se ejecuta durante la inicialización del componente. Es adecuado para la primera consulta porque el usuario aún no ha tenido ocasión de pulsar un botón. No equivale, sin embargo, a «consultar siempre que alguien necesite datos». Si el componente sigue vivo, un cambio producido en otro sistema no vuelve a llamar automáticamente a `OnInitializedAsync`.

Podríamos copiar las mismas instrucciones HTTP en el método de inicialización y en el manejador del botón. El resultado quizá funcionaría inicialmente, pero tendríamos dos sitios donde mantener los mensajes, el estado de carga, el tratamiento de errores y la lógica de actualización. La duplicación generaría un problema pedagógico y técnico: cada corrección posterior tendría que aplicarse en ambos caminos.

La alternativa es hacer que ambas entradas convoquen **una misma operación con nombre**:

```
protected override Task OnInitializedAsync() => ConsultarAsync();

private async Task ConsultarAsync()
{
    cargando = true;
    error = null;

    try
    {
        productos = (await CatalogoApi.ObtenerAsync()).ToList();
    }
    catch (HttpRequestException)
    {
        error = "No se pudo obtener el catálogo remoto.";
    }
    finally
    {
        cargando = false;
    }
}
```

El fragmento presupone campos de componente y un servicio ya registrados. Es ilustrativo y no pretende sustituir la implementación definitiva, que también trata cancelación y errores JSON. La transformación importante no es añadir más código sino **sacar una operación reutilizable** de un lugar que solo se ejecuta una vez.

Ahora comparemos dos recorridos. En el primero, Blazor inicializa el componente y llama a `ConsultarAsync`. En el segundo, el navegador envía el evento del botón y vuelve a llamar a `ConsultarAsync`. Después del punto de entrada, el flujo es idéntico: bandera de carga, servicio HTTP, asignación de lista, finalización. Un cambio en el método beneficia a ambos recorridos y evita que el botón se comporte de modo distinto al arranque.

En Interactive Server, el manejador del evento es código .NET ejecutado en el servidor, no JavaScript escrito por nosotros. La conexión interactiva mantiene la comunicación necesaria con el navegador. Este detalle importa cuando se depura: que un botón tenga `@onclick` no significa que se esté efectuando un `fetch` directamente desde el navegador.

5.1.12. Cuatro estados visuales que no deben confundirse

Un componente que consulta una API no tiene solamente los estados «funciona» y «falla». Al menos conviene distinguir **consulta pendiente**, **error**, **respuesta válida sin productos** y **respuesta válida con productos**.

Para expresar esas posibilidades mediante las propiedades ya conocidas se puede usar la siguiente condición conceptual:

```
@if (cargando)
{
    <p>Consultando el servidor...</p>
}
else if (error is not null)
{
    <p role="alert">@error</p>
}
else if (productos.Count == 0)
{
    <p>No hay productos disponibles.</p>
}
else
{
    <ul>
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre</li>
        }
    </ul>
}
```

La distinción entre `productos.Count == 0` y `error is not null` es esencial. Un servicio puede devolver correctamente `[]`; esa respuesta no indica una avería. Y un fallo de red puede ocurrir incluso cuando la última lista recibida contiene elementos. La interfaz necesita representar ambas realidades sin decir que el servidor está vacío cuando ni siquiera ha respondido.

Una respuesta HTTP 200 acompañada de `[]` satisface el contrato de una colección. Una respuesta HTTP 200 acompañada de `{"mensaje":"listo"}` **no** satisface necesariamente el contrato esperado por un cliente que deserializa `List<ProductoDto>`. El código de estado HTTP solo responde a una parte de la pregunta. El contenido tiene que ser coherente con el tipo que el servicio ha solicitado.

Finalmente, conviene identificar el problema del **estado anterior visible mientras se carga**. Algunos diseños sustituyen la lista por una indicación de carga; otros conservan la lista e incorporan un indicador discreto. Ambas decisiones pueden ser correctas según el uso. Lo que no es correcto es presentar sin matices una lista anterior como si acabara de confirmarse desde el

servidor. En el curso se adopta una solución sencilla: la consulta modifica las banderas de carga y error, y el componente ofrece una respuesta visual inequívoca.

5.1.13. El significado de `await` y las transiciones de estado

Una consulta asincrónica devuelve una tarea; no devuelve inmediatamente los productos. Cuando un método hace `await` sobre esa tarea, permite que el entorno gestione otras operaciones mientras llega la respuesta. Eso no quiere decir que el resultado pueda consumirse antes de que la tarea termine.

Examinemos el orden de instrucciones:

```
cargando = true;
error = null;

try
{
    var respuesta = await CatalogoApi.ObtenerAsync();
    productos = respuesta.ToList();
}
finally
{
    cargando = false;
}
```

La asignación a `productos` se ejecuta **después** de obtener `respuesta`. Si la consulta falla, esa asignación no se ejecuta. En cambio, el bloque `finally` se ejecuta tanto en una terminación satisfactoria como en una excepción. Así se evita dejar el botón inutilizable por una bandera `cargando` que permaneciera en `true` para siempre.

Los `catch` no son sustitutos del `finally`. Sirven para expresar qué error se observó y qué decisión toma el componente. La comprobación de que la operación terminó corresponde a un bloque que se ejecuta con independencia del resultado. Esta separación es una regla útil para otros ejemplos asincrónicos del curso: también la acción de crear tiene una bandera de envío que debe restaurarse.

Pregunta razonada. ¿Qué ocurriría si `cargando = false` estuviera exclusivamente después del `await`, fuera de un `finally`, y la llamada fallara? La ejecución abandonaría el bloque antes de llegar a esa línea. La interfaz podría seguir mostrando «Cargando...» e impedir nuevas pulsaciones aunque la solicitud ya hubiera terminado con un error. El fallo sería de gestión de estado local, no de la API.

Otra pregunta. ¿Por qué se borra el error anterior antes de consultar? Porque una nueva solicitud representa una oportunidad de recuperarse. Si el servidor había estado caído y vuelve a responder, no es coherente seguir mostrando la advertencia de la petición anterior después de recibir una lista satisfactoria.

5.1.14. La creación y el identificador pertenecen al servidor

Un formulario de creación recoge nombre y precio. La pantalla puede comprobar que el nombre no está vacío y que el precio es positivo, pero esa comprobación no crea un recurso en el servidor. Para ello hace falta una solicitud POST que el servidor valide y procese.

Imaginemos que una página muestra dos productos y decide asignar al siguiente el identificador `productos.Count + 1`, que sería 3. Si otro cliente ha creado un producto antes, el identificador 3

podría estar ocupado. Si se eliminara un elemento, la cuenta visible tampoco tendría por qué corresponderse con el siguiente identificador válido. En consecuencia, la interfaz no debe calcular el ID.

El patrón correcto para un alta es:

```
var entrada = new CrearProductoDto(nombre.Trim(), precio);
var creado = await CatalogoApi.CrearAsync(entrada);
productos.Add(creado);
```

Aquí `creado` es la representación devuelta por el servidor. El servicio HTTP debe comprobar primero que recibió una respuesta satisfactoria y deserializar el cuerpo esperado. Solo después de obtener ese objeto tiene sentido incorporarlo a la colección del componente.

Si el POST devuelve **400**, el componente no debería añadir el producto como si hubiese sido creado. Si la respuesta devuelve **201** pero el cuerpo JSON está incompleto, tampoco debería construir artificialmente el identificador. En ambos casos corresponde mostrar información suficiente para investigar y preservar la coherencia de la interfaz.

La diferencia entre recargar toda la lista después de POST y añadir el resultado del POST a la lista existente debe explicarse como una **decisión de sincronización**. Añadir el resultado es eficiente cuando se acaba de crear un recurso válido. Volver a consultar ofrece una instantánea más reciente del catálogo completo. En este curso se pueden observar ambas alternativas sin introducir sistemas de notificación en tiempo real.

5.1.15. Una lista en memoria y una base de datos SQL Server no comparten garantías

En M02 el proyecto MVC A utiliza EF Core y SQL Server Express LocalDB. La aplicación crea un contexto, realiza consultas y guarda cambios en una base de datos. Los datos persisten con independencia de que el proceso web termine, mientras la base esté disponible y no se elimine. En M04 y P5.1 la API docente mantiene un diccionario de productos dentro de su proceso.

Esta diferencia no es un detalle de instalación. Cambia lo que puede afirmarse sobre el resultado de una operación. Si un POST añade Regla a una colección estática en memoria, una consulta posterior al mismo proceso la encontrará. Sin embargo, al detener y reiniciar la API se inicializarán otra vez los valores de partida. Ese comportamiento es esperado, no una pérdida accidental que deba corregirse trasladando la API a SQLite.

Comparación causal:

Pregunta	API de productos en memoria	MVC con EF Core y LocalDB
¿Quién conserva la información?	El proceso ASP.NET Core	El motor SQL Server LocalDB
¿Qué sucede al reiniciar solo el proceso web?	Se recrea la colección inicial	Los datos de la base siguen disponibles
¿Hace falta instalar LocalDB para GET y POST de P5.1?	No	No aplica a esa API
¿Se puede afirmar persistencia por recibir 201?	No	La persistencia se comprueba según el flujo EF

Pregunta	API de productos en memoria	MVC con EF Core y LocalDB
¿Es adecuada para la demostración de despliegue Linux de M05?	No se usa allí	El CRUD de MVC requiere SQL Server real

La última fila ayuda a entender una restricción posterior: en el contenedor Docker del curso se publica MVC A, pero **no se instala LocalDB dentro del contenedor Linux**. El ejercicio demostrará que el proceso MVC y sus recursos básicos se sirven; no fingirá que el CRUD con base de datos está disponible allí.

5.1.16. Recorrido de diagnóstico en seis preguntas

Cuando una operación falla, una secuencia de preguntas reduce las modificaciones improvisadas:

Primera: ¿existe el proceso que debería atender la solicitud? Ver un navegador abierto no demuestra que siga vivo el proceso de API. Consultar su consola y su URL directa permite distinguir la ausencia de servicio de un error del componente.

Segunda: ¿se utiliza la dirección correcta? `http://localhost:5081/` identifica en el ejemplo el servidor API; `http://localhost:5092/` identifica Blazor. Una respuesta de la página inicial Blazor no demuestra que el endpoint de productos haya recibido la solicitud. Deben verificarse host, puerto y ruta.

Tercera: ¿la API contesta y con qué estado? Un 404 demuestra que el servidor recibió la solicitud, pero no encontró el recurso solicitado. Un error de conexión significa que no se consiguió una respuesta HTTP equivalente. Son hechos distintos y requieren actuaciones distintas.

Cuarta: ¿el cuerpo responde al contrato? Un 200 puede contener `[]`, una lista de productos válida o un texto que no es JSON. Leer el cuerpo ayuda a separar fallo de transporte y fallo de interpretación.

Quinta: ¿el servicio transforma correctamente el resultado? Si la API devuelve datos válidos y Blazor sigue mostrando un error, revisar el servicio tipado, su configuración de dirección base y la excepción concreta. No duplicar rutas HTTP en el componente para eludir la investigación.

Sexta: ¿la interfaz está presentando el estado recibido? Aunque la red funcione, una colección no actualizada o una bandera de carga incorrecta pueden hacer que la página parezca rota. Examinar el método que asigna `productos` y el orden de condiciones Razor suele resultar más útil que añadir un nuevo proyecto.

El orden importa. Empezar por cambiar la UI sin comprobar el servidor puede ocultar una avería externa; empezar por la API sin considerar el estado del componente puede atribuir al servidor un fallo de renderizado. La clave es aislar primero la capa donde el resultado deja de coincidir con el contrato.

5.1.17. Caso integral resuelto: el producto que parece desaparecer

Una persona crea Regla desde Blazor. La página recibe un DTO con ID asignado por la API y lo añade a la lista. Todo parece correcto. Después, el docente detiene y vuelve a iniciar la API. En la pantalla anterior todavía figura Regla. Al pulsar «Consultar servidor» desaparece.

¿Por qué se produce este comportamiento? El POST creó el producto en memoria, no en SQL Server. El reinicio volvió a poblar la colección inicial. La pantalla mantenía una representación

anterior hasta ejecutar una consulta nueva. La actualización no eliminó por error un registro persistente: mostró el contenido del nuevo proceso.

¿Qué observaciones permitirían confirmar la explicación? Antes del reinicio, un GET directo a la API devuelve Regla. Después del reinicio, el mismo GET devuelve solo los productos iniciales. Mientras tanto, antes de recargar Blazor, la pantalla puede conservar Regla. Al consultar explícitamente, la UI coincide con la última respuesta. Se han identificado tres estados y una secuencia de eventos sin introducir herramientas ajenas al temario.

¿Qué corrección no deberíamos aplicar? No deberíamos modificar MVC A ni sustituir LocalDB por SQLite ni incorporar Identity/JWT a esta API para conseguir persistencia. La API en memoria cumple exactamente el papel docente del módulo. Si se quisiera implementar persistencia duradera en otro curso, habría que plantear explícitamente otro contrato, otra infraestructura y nuevas pruebas. Eso no forma parte del objetivo de P5.1.

Conclusión. La integración queda comprendida cuando el alumno puede explicar no solo la respuesta correcta, sino también una secuencia aparentemente contradictoria de respuestas correctas obtenidas en instantes distintos.

5.1.18. El contrato entre servicio y componente: comprender un resultado sin ver HTTP en la vista

Hasta aquí hemos separado el componente de la API. Conviene examinar una consecuencia concreta de esa separación: la vista puede trabajar con `IReadOnlyList<ProductoDto>` sin saber cómo se ha recibido el JSON. Esta decisión facilita la lectura y el mantenimiento, pero requiere que el servicio encapsule de verdad la responsabilidad HTTP.

El contrato utilizado en la aplicación B presenta dos operaciones fundamentales:

```
public interface IProductosApiService
{
    Task<IReadOnlyList<ProductoDto>> ObtenerAsync();
    Task<ProductoDto> CrearAsync(CrearProductoDto solicitud);
}
```

La primera promete una tarea que, al finalizar con éxito, proporcionará una colección de productos. La segunda promete el producto que el servidor haya creado. Ni el método de consulta ni el de creación necesitan recibir referencias a botones, textos HTML o componentes Razor. El servicio no sabe dónde se dibuja un nombre: su trabajo concluye al entregar el dato o manifestar un error.

Primer razonamiento: ¿por qué el método de creación devuelve `ProductoDto` y no simplemente `bool`? Porque el servidor decide un identificador y puede normalizar información. Una respuesta afirmativa aislada perdería la representación creada. El componente necesita ese resultado para actualizar su colección sin inventar un ID.

Segundo razonamiento: ¿por qué una tarea asincrónica? Porque la respuesta depende de una operación externa que puede tardar o fallar. Un contrato sincrónico que bloqueara innecesariamente la ejecución resultaría menos adecuado para un componente que debe representar una operación en curso.

Tercer razonamiento: ¿por qué una interfaz? Porque expresa lo que el componente necesita y permite cambiar detalles de implementación o de configuración HTTP sin trasladar la URL a cada elemento visual. No es necesario introducir una arquitectura empresarial compleja; con dos operaciones y un servicio concreto basta para este curso.

5.1.19. Leer la implementación del servicio desde las operaciones HTTP

El método de lectura del proyecto utiliza una forma compacta del cliente HTTP tipado:

```
public async Task<IReadOnlyList<ProductoDto>> ObtenerAsync() =>
    await cliente.GetFromJsonAsync<List<ProductoDto>>("api/productos") ?? [];
```

Esta expresión concentra tres ideas. Primero, `cliente` ya debe haber recibido su dirección base durante el registro de servicios. Segundo, la ruta relativa `api/productos` se combina con esa dirección para formular la solicitud. Tercero, la respuesta JSON se transforma en una lista de objetos `ProductoDto`; el componente no tiene que realizar la deserialización por su cuenta.

El operador `?? []` cubre el caso en que la operación de deserialización produzca una referencia nula. No convierte en válidos cuerpos JSON arbitrarios. Si el contenido no puede convertirse al tipo esperado, la deserialización puede producir una excepción. La distinción es significativa: «no hay objeto» y «el JSON está mal formado» son estados técnicos distintos.

La operación de creación muestra más pasos porque necesita enviar el cuerpo y conservar la representación del recurso creado:

```
public async Task<ProductoDto> CrearAsync(CrearProductoDto solicitud)
{
    using var respuesta = await cliente.PostAsJsonAsync("api/productos", solicitud);
    respuesta.EnsureSuccessStatusCode();

    return await respuesta.Content.ReadFromJsonAsync<ProductoDto>()
        ?? throw new InvalidOperationException(
            "La API confirmó un alta sin devolver un producto.");
}
```

`PostAsJsonAsync` envía un cuerpo JSON a la ruta indicada. `EnsureSuccessStatusCode` impide tratar una respuesta 400 como si representara un alta correcta. `ReadFromJsonAsync` intenta convertir la respuesta en el DTO esperado. Si el servidor no devuelve ese objeto, el servicio no debe fabricar una representación que no existe.

El `using` indica que el objeto de respuesta HTTP se libera al salir del método. No significa que se cierre inmediatamente todo el cliente HTTP tipado ni que la aplicación deje de poder realizar futuras llamadas. Es importante no confundir el ciclo de vida de la respuesta concreta con la infraestructura administrada por `HttpClientFactory`.

5.1.20. Interpretar correctamente el flujo de excepciones

Supongamos cuatro situaciones.

En la primera, el servidor no escucha la dirección configurada. La consulta no consigue una respuesta HTTP útil. El componente puede recibir una excepción de transporte y representar «No se pudo obtener el catálogo remoto». Este resultado no demuestra que la ruta sea incorrecta; tal vez el proceso esté detenido o el puerto difiera.

En la segunda, el servidor responde 404 para una ruta. El cliente HTTP ha conseguido comunicarse. Si la operación utiliza métodos que exigen respuesta satisfactoria, puede producirse una `HttpRequestException` asociada a ese código, pero la investigación debe distinguirla de una conexión rechazada.

En la tercera, el servidor responde 200 con un arreglo JSON vacío. La solicitud y la deserialización se completan. El componente recibe una lista sin elementos; debe representarla como **ausencia válida de productos** y no como avería de comunicación.

En la cuarta, el servidor devuelve 200 con un texto que declara JSON pero no puede analizarse como una lista. El servicio intenta deserializar y puede producirse `JsonException`. La consulta llegó al servidor; el fallo aparece al interpretar el contrato de contenido.

Por ese motivo el código del componente contempla clases de errores distintas. No pretende adivinar la causa exacta de cualquier fallo de red a partir de un único texto, pero sí evita un mensaje indistinto que llevaría al alumno a modificar la capa equivocada.

5.1.21. La importancia del orden de las condiciones Razor

En una interfaz con operaciones asincrónicas, el orden de renderizado expresa una política. Esta secuencia es especialmente instructiva:

```
@if (cargando)
{
    <p role="status">Consultando...</p>
}
else if (error is not null)
{
    <p role="alert">@error</p>
}
else if (productos.Count == 0)
{
    <p>La consulta terminó, pero no hay productos.</p>
}
else
{
    <ul>
        @foreach (var producto in productos)
        {
            <li>@producto.Nombre</li>
        }
    </ul>
}
```

La primera rama representa una consulta en curso. La segunda indica que no podemos presentar la respuesta como satisfactoria. La tercera describe una respuesta válida sin elementos. Solo la cuarta presenta una colección con productos. Cambiar el orden puede introducir mensajes engañosos aunque el servicio HTTP sea correcto.

Caso de error: una consulta falla antes de haber recibido productos y la colección inicial permanece vacía. Si la comprobación de colección vacía precede a la de `error`, la pantalla podría informar de que el servidor no tiene productos. En realidad no hemos obtenido su representación. La explicación correcta reside en la precedencia de las condiciones.

Caso de éxito vacío: la API devuelve `[]`. No se ha producido excepción; la colección está vacía después de una respuesta correcta. Si la pantalla usa la misma frase para esta situación y para una caída de red, pierde una información fundamental para el usuario y para el aprendizaje.

La lógica de presentación debe corresponder a los hechos conocidos por el componente, no a suposiciones sobre el estado del servidor.

5.1.22. La bandera del botón reduce duplicaciones, pero no es un sistema de concurrencia distribuida

Deshabilitar el botón durante una consulta es una decisión útil:

```
<button disabled="@cargando" @onclick="ConsultarAsync">
    Consultar servidor
</button>
```

Evita que el usuario lance desde ese mismo control otra operación mientras se representa la carga. Sin embargo, no significa que toda la aplicación quede bloqueada ni que distintos clientes de la API se coordinen a través de esa bandera.

Un segundo navegador puede crear un producto mientras el primer componente está consultando. El servidor API sigue atendiendo solicitudes de clientes diferentes. Además, el componente podría disponer de otros eventos capaces de cambiar su estado. Por tanto, no debemos afirmar que `disabled` garantiza un orden global entre todas las operaciones HTTP.

El laboratorio adopta un comportamiento deliberadamente sencillo: la pantalla informa de carga y evita pulsaciones repetidas desde el botón. No desarrolla control de concurrencia distribuida ni sincronización en tiempo real. Entender este límite es suficiente para no atribuir a la interfaz garantías que corresponden a otras capas.

5.1.23. Una operación puede producir dos efectos observables distintos

Cuando el alumno pulsa «Crear», ocurren al menos dos cosas diferenciadas. Por un lado, se modifica la colección mantenida por la API si el POST es válido. Por otro, el componente añade localmente el DTO devuelto y actualiza la interfaz.

La segunda acción no demuestra por sí sola la primera si ignoramos el contrato del servidor. Por ello conviene contrastar la pantalla con un GET externo. A la inversa, un POST realizado desde otra terminal puede cambiar la colección de la API sin que la pantalla reciba inmediatamente esa información.

Esta doble observación sirve para explicar por qué los componentes de interfaz son representaciones temporales de un estado externo. En la práctica P5.1, la operación `ConsultarAsync` permite actualizar esa representación bajo demanda.

La distinción es también un criterio de diagnóstico: si POST devuelve 201 pero el producto no aparece en la pantalla, revisaremos el modo en que el componente incorpora el DTO. Si el producto aparece localmente pero no se encuentra mediante GET, deberemos investigar qué servidor respondió y si el contrato fue realmente satisfecho. No se resuelve ninguno de los dos casos reescribiendo sin motivo toda la API.

5.1.24. Un ensayo reproducible necesita restaurar el servidor

Para comparar lista normal, lista vacía y JSON inválido se puede modificar temporalmente la respuesta del mismo endpoint. Eso permite estudiar fallos realistas, pero no debe convertirse en el estado permanente de la aplicación que recibe el siguiente alumno.

La variante de diagnóstico del curso elige un escenario por configuración y preserva el comportamiento normal como valor predeterminado. Al acabar, se restaura el `Program.cs` original y se elimina la variable que había forzado el escenario. Así la API vuelve a sus dos productos iniciales y a su contrato habitual.

La restauración forma parte de la competencia práctica: permite repetir el mismo experimento, comparar los resultados y evitar que una modificación temporal se convierta en una dependencia oculta. No es una operación meramente administrativa; es la condición que hace que las explicaciones y las instrucciones mantengan su validez para todo el alumnado.

5.1.25. Diferenciar un error de transporte de una respuesta 404

Una conexión rechazada y una respuesta HTTP 404 no son el mismo resultado. En el primer caso puede no existir respuesta HTTP porque el proceso está detenido o porque la dirección no resulta accesible. En el segundo, un servidor ha recibido la petición y ha respondido que el recurso solicitado no existe.

El servicio HTTP de la aplicación utiliza `GetFromJsonAsync`. Si recibe un código HTTP no satisfactorio, puede producir una `HttpRequestException` cuyo campo `StatusCode` identifica el estado de la respuesta. Cuando no se ha obtenido una respuesta HTTP, esa propiedad puede ser nula.

Podemos tratar primero el caso conocido y después conservar una respuesta general para el resto:

```
catch (HttpRequestException ex)
    when (ex.StatusCode == System.Net.HttpStatusCode.NotFound)
{
    error = "La API respondió 404: no existe la ruta del catálogo.";
}
catch (HttpRequestException)
{
    error = "No se pudo obtener el catálogo remoto.";
}
```

El orden es importante. Una captura general situada antes capturaría también la excepción 404 y nunca se alcanzaría la condición especializada. Estos bloques presuponen el método de consulta y el campo `error` del componente ya explicados. No constituyen otro proyecto.

5.1.26. Comparar cuatro resultados sin inventar la causa

Resultado	Qué sabemos	Qué no debemos afirmar
Conexión rechazada	No se obtuvo una respuesta HTTP útil	Que el servidor contestó 404
HTTP 404	La ruta o recurso solicitado no existe en ese servidor	Que no había ningún servidor escuchando
HTTP 500	El servidor no completó satisfactoriamente la solicitud	Que el cuerpo JSON estaba necesariamente mal formado
HTTP 200 con JSON inválido	La respuesta HTTP fue satisfactoria	Que la deserialización tuvo éxito

La mejora en el componente es limitada y pedagógica: permite explicar el caso 404 con mayor precisión. No pretende deducir automáticamente cualquier causa de un error 500. Para ello se necesitarían los mensajes y el contexto del servidor.

5.1.27. Un sistema de cuatro procesos conceptuales que no deben confundirse

Al integrar la aplicación Blazor B con la API de productos, es tentador hablar de «la aplicación» como si existiese una sola entidad. En realidad, incluso en un laboratorio local intervienen un navegador, el host ASP.NET Core que ejecuta Blazor, el host ASP.NET Core que ejecuta la API y, opcionalmente, herramientas externas de prueba como PowerShell. Cada uno posee responsabilidades propias. La ventana del navegador recibe HTML y mantiene la interacción; el servidor Blazor ejecuta el componente Interactive Server; el servicio HTTP tipado crea solicitudes dirigidas a la API; la API decide cómo responder. El segundo servidor no tiene por qué estar vivo para que el primero entregue su página inicial.

Podemos estudiar un caso en el que Blazor continúa funcionando mientras la API está detenida. Al abrir la ruta `/catalogo-remoto`, el host Blazor genera el componente y trata de consultar el catálogo. Si la conexión a 5081 falla, el componente puede mostrar un mensaje de error. La presencia del mensaje demuestra precisamente que hubo suficiente ejecución Blazor como para representar el estado de fallo. Sería contradictorio concluir que todo Blazor está apagado cuando el propio componente ha comunicado la excepción.

Un segundo caso invierte la situación: la API responde 200 a una consulta directa, pero el navegador no logra abrir 5092 porque Blazor está detenido. La API funciona, y eso no abre automáticamente una interfaz Blazor. La solución no consiste en cambiar el controlador de productos de la API. Hace falta iniciar el host que presenta los componentes.

Un tercer caso presenta los dos hosts activos, pero la configuración de `ProductosApi:BaseUrl` apunta a 5082 en lugar de 5081. Las dos aplicaciones responden por separado, pero la comunicación entre ellas falla. El primer diagnóstico razonable es comprobar la URL que utiliza el servicio, no reescribir `ProductoDto`.

Estas tres situaciones comparten el mismo escenario de aula y producen fallos visuales diferentes. Identificar **qué proceso está vivo, qué dirección utiliza cada uno y dónde se interrumpe la comunicación** es más importante que aprender una lista indiscriminada de posibles errores.

5.1.28. El componente no se convierte en cliente HTTP por contener un botón

El botón «Consultar servidor» enlaza un evento interactivo con un método del componente. Ese método delega en `IProductosApiService`, cuyo servicio concreto emplea `HttpClient`. No debe trasladarse la lógica de direcciones, serialización y códigos de estado al HTML de la página.

La frontera resulta visible en un ejemplo conceptual de dos responsabilidades. En el componente:

```
private async Task ConsultarAsync()
{
    cargando = true;
    try
    {
        productos = (await CatalogoApi.ObtenerAsync()).ToList();
        error = null;
    }
    catch (HttpRequestException)
    {
        error = "No fue posible recuperar los productos.";
    }
    finally
    {
```

```
        cargando = false;
    }
}
```

En el servicio:

```
public async Task<IReadOnlyList<ProductoDto>> ObtenerAsync()
{
    var datos = await cliente.GetFromJsonAsync<List<ProductoDto>>(
        "api/productos");
    return datos ?? [];
}
```

Ambos fragmentos presuponen sus tipos y registros de inyección de dependencias. El primero habla de estados visibles; el segundo de transporte y datos. Si la API cambia de puerto, corregiremos la dirección base del cliente, y no añadiremos una nueva URL dentro del botón. Si cambia el texto de carga, modificaremos la vista, y no el método HTTP.

Esta separación evita una confusión especialmente frecuente al comienzo: pensar que reutilizar un servicio significa duplicar la misma llamada HTTP en cada componente. Reutilizar significa conservar la operación en un único lugar y llamarla desde los consumidores pertinentes.

5.1.29. El estado inicial de un componente no es un resultado de la API

Cuando un componente declara `private List<ProductoDto> productos = [];`, crea una colección inicialmente vacía. Esa instrucción no afirma nada sobre el catálogo remoto: se ejecuta incluso cuando el servidor API dispone de cientos de productos. Una lista inicial vacía permite construir el componente antes de completar la primera solicitud, pero no equivale a la respuesta HTTP `[]`.

La bandera `cargando` ayuda a expresar esa incertidumbre. Durante la consulta inicial, la interfaz debe comunicar que todavía no dispone de datos confirmados. Después de una respuesta 200 con lista vacía, esa incertidumbre cambia: ahora sí sabemos que el servidor ha contestado con una colección sin elementos. En cambio, después de un error HTTP no podemos tratar la colección inicial como prueba de ausencia de productos.

La diferencia puede representarse como una máquina de estados conceptual:

```
Inicialización
      |
      v
Consultando ---- error de red/HTTP/JSON ---> Error
      |
      +---- respuesta [] -----------------> Vacío válido
      |
      +---- respuesta [datos] ------------> Datos visibles
                      ^
                      |
                Nueva consulta
```

El diagrama simplifica el comportamiento: durante la siguiente consulta puede existir una representación anterior. Esa representación no deja de existir automáticamente cuando empieza una petición. La interfaz decide si la oculta mediante la rama de carga o si la mantiene visible con una indicación de actualización.

El contrato del curso utiliza un orden explícito de comprobaciones: carga, error, vacío válido y datos. Reconocer el orden permite explicar por qué modificar una condición Razor cambia el significado del mensaje sin alterar la respuesta de la API.

5.1.30. Un resultado externo requiere una operación observable de confirmación

Si el alumno crea un producto desde Blazor y lo ve aparecer, tiene una evidencia inicial de que el componente ha recibido una representación satisfactoria. Pero una práctica completa necesita comprobar el servidor de forma independiente. Un GET directo a `/api/productos` después del POST permite contrastar lo que muestra la pantalla con lo que conserva la API.

Es razonable preguntar qué sucedería si el método `CrearAsync` añadiera artificialmente un objeto local antes de enviar HTTP. La pantalla podría mostrar un producto que el servidor jamás recibió. Por eso el orden correcto del ejemplo real es enviar, comprobar el estado satisfactorio, interpretar el DTO recibido y entonces añadirlo a la lista visual.

El ejercicio de creación desde `curl.exe` invierte el recorrido: el POST externo añade información al servidor sin que el componente Blazor participe. La página original no recibe una notificación automática. Solo una nueva consulta trae la información a la colección local. Con dos observaciones opuestas demostramos las dos direcciones de la separación: el componente no debe inventar el estado remoto y el servidor no modifica automáticamente los campos de un componente que no ha vuelto a consultar.

Esta técnica es transferible a escenarios más grandes sin enseñarlos aquí: en cuanto existe un sistema externo, una actualización visual no sustituye una comprobación independiente de la fuente.

5.1.31. Un identificador creado por el servidor no se puede deducir contando filas

El producto creado por POST recibe un identificador a cargo del servidor. En la API docente, un contador atómico produce un nuevo valor; el cliente recibe una respuesta que contiene el identificador asignado. La interfaz no debe adivinarlo usando la longitud de la colección.

Supongamos una lista visual de dos productos. Un cliente externo crea uno y obtiene ID 3. Otro cliente abre la pantalla antigua, ve solo dos elementos e intenta asignar el siguiente ID mediante `Count + 1`. El resultado también sería 3, aunque ya está ocupado. Si alguien eliminara un recurso en otro sistema, contar elementos tampoco produciría necesariamente un identificador libre.

La consecuencia es sencilla: `CrearProductoDto` contiene los campos de entrada, mientras que `ProductoDto` de respuesta incluye la identidad asignada. El componente recibe ese resultado, no calcula la identidad. Aunque el laboratorio sea pequeño, esta responsabilidad no debe confundirse.

El servidor también normaliza el nombre con `Trim()`. Enviar una cadena con espacios al principio y al final puede producir un recurso cuya representación devuelta no es idéntica carácter a carácter al formulario original. Por eso conviene utilizar el DTO resultante en lugar de reconstruirlo a partir del texto que escribió el usuario.

5.1.32. El significado de 201 y la necesidad de leer el cuerpo

La respuesta 201 Created confirma que el servidor ha atendido satisfactoriamente una creación según el contrato HTTP del endpoint. En la API del curso, el resultado contiene la representación

creada y una ubicación del recurso. No significa que la información haya sido guardada en SQL Server, porque la API de referencia usa memoria.

Un cliente de creación tiene que interpretar el resultado con cuidado. Primero verifica que el estado es satisfactorio. Después intenta leer el `ProductoDto`. Si el servidor devolviese un 201 sin cuerpo apropiado, el código del servicio no debería inventar un producto para actualizar la lista. La implementación muestra una excepción `InvalidOperationException` cuando el valor deserializado resulta nulo.

Una respuesta 400 durante POST describe una entrada que incumple el contrato del servidor. No se debe añadir localmente un producto como si se hubiera creado. Una respuesta 500 describe una incidencia del servidor, no una garantía de que la operación se completó. Una interrupción de red durante POST plantea incluso una ambigüedad: no siempre sabemos si el servidor procesó la solicitud antes de que el cliente perdiera la conexión. El laboratorio no implementa reintentos automáticos de creación, precisamente porque repetir ciegamente un POST podría duplicar una operación cuando no tenemos confirmación suficiente.

Este último matiz excede la implementación mínima, pero es fundamental para razonar sobre resultados inciertos: **una excepción del cliente no demuestra en todos los casos que el servidor no hizo nada**.

5.1.33. Errores de validación local y de validación de servidor

El formulario puede rechazar una entrada sin nombre o con precio no positivo antes de enviar HTTP. Esa comprobación mejora la experiencia y evita solicitudes inútiles. Pero el endpoint debe repetir su propia validación para no depender de la honestidad de un único cliente.

La prueba con `curl.exe` demuestra este límite. Un cliente externo no ejecuta las condiciones C# del componente Blazor. Si envía un cuerpo inválido, el servidor debe contestar 400. Si la API aceptara el dato por depender solo del formulario, su contrato sería incorrecto.

El alumno debe saber distinguir el origen de los dos mensajes. Un rechazo local puede representarse sin que la API haya recibido ningún POST. Un rechazo de servidor es una respuesta HTTP observable directamente desde otro cliente. Aunque ambas decisiones utilicen las mismas reglas básicas de nombre y precio, pertenecen a capas distintas.

Este principio también aclara por qué la autenticación y la autorización del módulo anterior no se sustituyen ocultando controles visuales. Las decisiones que protegen un recurso deben aplicarse del lado del servidor. La interfaz puede adaptar lo que muestra, pero no debe ser la única barrera que impida operaciones no autorizadas.

5.1.34. Una consulta correcta puede presentar información temporalmente obsoleta

Tras un GET satisfactorio, los datos representados son correctos respecto a la respuesta recibida en ese instante. No necesariamente permanecerán actuales para siempre. En la API de memoria, un segundo cliente puede añadir productos; la primera página conserva la última lista que obtuvo.

No debemos tratar este comportamiento como si Blazor hubiera perdido una señal obligatoria del servidor. El contrato del curso no introduce una conexión de notificaciones ni sincronización automática. El usuario pulsa «Consultar servidor» para obtener otra instantánea. Es una decisión didáctica: permite observar la diferencia entre datos locales y datos remotos sin incorporar SignalR personalizado o una infraestructura de eventos adicional.

Si el nuevo GET falla, puede permanecer en memoria la lista antigua, pero la pantalla utiliza su rama de error. Si el GET termina con `[]`, la lista se sustituye por una colección vacía y la interfaz lo indica. Ambos comportamientos son coherentes con la implementación; las conclusiones cambian según el resultado obtenido.

El aprendizaje esperado es expresar con precisión **cuándo se obtuvo la información**, **de dónde procede** y **qué operación la actualiza**. Esas preguntas evitan diagnósticos equivocados sobre persistencia y sobre renderizado.

5.1.35. La ubicación de una llamada HTTP en Blazor Interactive Server

En los módulos anteriores se aprendió que una página Razor puede contener código C# que se ejecuta en el servidor. En Blazor Interactive Server ese principio se extiende a componentes que atienden eventos del usuario. Es fácil que el alumno vea un botón y suponga que toda llamada HTTP procede directamente del navegador. En nuestro proyecto no es así: el componente se ejecuta en el servidor Blazor, y el servicio tipado utiliza `HttpClient` desde ese proceso para comunicarse con la API.

La ruta física de una solicitud de catálogo se describe mejor así:

```
Navegador: clic en Consultar servidor
              |
              v
Blazor Interactive Server: recibe el evento
              |
              v
Componente: ConsultarAsync
              |
              v
IProductosApiService / ProductosApiService
              |
              v
HttpClient del servidor Blazor
              |
              v
HTTP GET a la API en localhost:5081
              |
              v
API: JSON y código de estado
              |
              v
Blazor actualiza el estado del componente
              |
              v
Navegador recibe la representación actualizada
```

Esta cadena justifica que el proyecto de aula ejecute dos servidores ASP.NET Core distintos. El proceso de Blazor no se transforma en API de productos porque consuma un endpoint HTTP, y el servidor API no se convierte en interfaz Blazor porque responda JSON.

El modelo también orienta el diagnóstico: si una dirección funciona en el navegador del alumno, no se ha demostrado automáticamente que un proceso Blazor alojado en otra máquina pueda alcanzarla. Cada proceso interpreta las direcciones y sus condiciones de red desde su propio entorno.

5.1.36. Por qué `localhost` debe interpretarse desde el proceso que consulta

Durante el laboratorio todos los servidores se ejecutan en la misma máquina del alumno, y por eso `http://localhost:5081/` es una dirección sencilla para la API. El nombre `localhost` siempre se interpreta respecto al contexto de red que realiza la conexión. No significa una dirección universal de un servidor disponible desde cualquier equipo.

En el modelo Interactive Server, el `HttpClient` utilizado por el componente pertenece al proceso de Blazor. Ese proceso necesita poder resolver y conectar con `localhost:5081`. Si Blazor se trasladara a otra máquina y la API permaneciera en la original, la misma dirección dejaría de señalar al servicio deseado. Habría que configurar una dirección alcanzable desde el nuevo proceso.

Este razonamiento es pertinente también para Docker: dentro de un contenedor, `localhost` identifica el propio contenedor. Por tanto, no debemos copiar indiscriminadamente una dirección de API local de Windows dentro de una imagen Linux y esperar que siga llegando al mismo destino. En M05 Docker utiliza solamente MVC A y evita ejecutar el cliente Blazor en ese entorno, precisamente para mantener un alcance de despliegue elemental.

5.1.37. CORS y solicitudes de servidor: no confundir dos escenarios

CORS regula determinadas solicitudes HTTP realizadas por código de navegador entre orígenes distintos. En el recorrido principal de Blazor Interactive Server, el servicio tipado realiza una solicitud **desde el servidor Blazor hacia la API**, no un `fetch` de JavaScript del navegador a `localhost:5081`.

Por ello, ante una conexión fallida desde el servicio tipado, no debemos afirmar que hace falta configurar CORS sin comprobar primero desde dónde se está realizando la solicitud. El problema podría ser un puerto incorrecto, una API detenida o una ruta HTTP no existente.

Si diseñáramos un cliente WebAssembly que llamara directamente a la API desde el navegador, las condiciones serían diferentes y CORS podría intervenir. Ese no es el modelo elegido para esta práctica y no se añade como tercer modo Blazor. La finalidad de la comparación es evitar un diagnóstico equivocado, no ampliar el curso a otra arquitectura.

5.1.38. La dirección base y la ruta relativa constituyen una URL completa

El registro del cliente tipado establece una `BaseAddress`. Los métodos del servicio utilizan rutas relativas como `api/productos`. Para que el resultado sea coherente debemos comprender cómo se combinan esas partes.

Con la base `http://localhost:5081/` y la ruta `api/productos`, el destino es `http://localhost:5081/api/productos`. El separador final de la dirección base y la ausencia de una barra inicial en la ruta ayudan a expresar que la llamada se realiza bajo la raíz de ese servidor.

En URI relativas existen reglas de resolución que pueden producir resultados inesperados cuando se omite la barra final de una dirección base con subruta o se utiliza una ruta absoluta que reemplaza parte de la base. El proyecto se mantiene deliberadamente sencillo y no explora todos los casos de resolución URI; utiliza una base local terminada en barra y una ruta de endpoint sin barra inicial.

Caso resuelto: si la API escucha en 5081 y la configuración del cliente contiene 5082, el código del método `ObtenerAsync` podría ser perfectamente correcto, pero no llegará al servidor previsto. La

corrección corresponde a la fuente de configuración de `BaseAddress`, no a duplicar la URL dentro del componente.

5.1.39. El registro del cliente tipado concentra decisiones comunes

La aplicación B utiliza inyección de dependencias para asociar `IProductosApiService` con `ProductosApiService`. El registro proporciona un `HttpClient` configurado y permite que el componente pida el servicio por su contrato.

Una versión ilustrativa del registro es:

```
builder.Services.AddHttpClient<IProductosApiService, ProductosApiService>(
    cliente =>
    {
        cliente.BaseAddress = new Uri("http://localhost:5081/");
        cliente.Timeout = TimeSpan.FromSeconds(10);
    });
```

En el proyecto real la dirección se obtiene de configuración. El objetivo del fragmento es mostrar que el timeout y la dirección base se establecen para el cliente que realizará las solicitudes, no en el marcado HTML de cada botón.

El componente no necesita instanciar `new HttpClient()` cada vez que el usuario consulta el catálogo. La fábrica de clientes HTTP utilizada en ASP.NET Core administra la infraestructura correspondiente y evita que cada componente duplique la preparación del transporte. El alumno no tiene que aprender aquí su configuración avanzada: basta con reconocer la responsabilidad del registro y el contrato que consume la vista.

5.1.40. El evento del botón y el ciclo de vida no son momentos equivalentes

`OnInitializedAsync` inicia la consulta cuando el componente se inicializa. `@onclick` vincula una acción del usuario con un método que se ejecuta posteriormente. Ambos pueden delegar en `ConsultarAsync`, pero representan momentos diferentes.

Una primera consulta puede producir dos productos. Una creación externa puede modificar el servidor sin afectar la instancia actual del componente. Solo cuando el usuario pulsa «Consultar servidor» se inicia otra operación de lectura. Este hecho prueba que el ciclo de vida no actúa como un mecanismo de actualización permanente.

Si se duplicara el código de consulta dentro de `OnInitializedAsync` y del manejador del botón, podrían aparecer diferencias en los mensajes de error o el estado de carga. Delegar ambos en una operación reutilizable preserva un único comportamiento y facilita probarlo.

En un componente interactivo, la finalización de un evento asincrónico provoca el proceso de renderizado correspondiente. No es necesario enseñar a llamar indiscriminadamente a `StateHasChanged` en cada línea como condición para que un botón funcione. El ejemplo conserva el modelo de actualización de Blazor que se introdujo en M03.

5.1.41. Qué significa desactivar la prerrepresentación en esta página

La directiva de la página utiliza:

```
@rendermode @(new InteractiveServerRenderMode(prerender: false))
```

El modo Interactive Server permite ejecutar eventos del componente en el servidor. La opción `prerender: false` evita la fase de prerrepresentación previa a la interactividad para este componente. En esta demostración facilita razonar sobre una inicialización que se realiza dentro del ciclo interactivo.

Esto no significa que la página deje de utilizar ASP.NET Core ni que su HTML aparezca mágicamente en el navegador. Significa que hemos elegido un comportamiento concreto del ciclo de vida y de su representación inicial.

El curso no transforma la página en WebAssembly ni introduce JavaScript propio para consumir la API. La explicación es importante para que un alumno no trate de corregir una consulta duplicada cambiando sin fundamento el endpoint HTTP o el componente de datos. Primero hay que reconocer el modo de renderizado y el ciclo de vida configurados.

5.1.42. Un cliente HTTP puede terminar por éxito, error o cancelación

Una solicitud asincrónica no devuelve siempre una colección. Puede finalizar correctamente con datos, finalizar correctamente con una colección vacía, devolver un error HTTP o fallar en transporte o deserialización. También puede verse afectada por el timeout configurado en `HttpClient`.

Por eso `ConsultarAsync` no debe cambiar el estado visual únicamente en la rama satisfactoria. Necesita iniciar la bandera de carga, capturar los errores que el laboratorio distingue y restaurar esa bandera en `finally`. El orden establece una correspondencia entre el estado técnico y lo que el usuario observa.

Un timeout no es lo mismo que un 404. En el primero podría no haberse obtenido una respuesta HTTP útil dentro del plazo. En el segundo un servidor atendió y contestó explícitamente que no existe la ruta o recurso. La interfaz presenta mensajes distintos cuando dispone de esa información.

Pregunta de comprensión: si la API tarda más que el tiempo admitido, ¿debe mostrar la interfaz «el catálogo está vacío»? No: sin una colección satisfactoria no conocemos el contenido actual del servidor.

5.1.43. Contar elementos visibles es una operación local

En la ampliación práctica se calcula el número de productos y la suma de precios sobre la colección `productos` del componente. Esta operación no ejecuta SQL ni una solicitud GET adicional. Sus resultados se obtienen de la representación local que Blazor mantiene.

El ejemplo sirve para visualizar un hecho que de otro modo puede pasar inadvertido: el servidor remoto puede haber cambiado mientras el contador sigue mostrando el número anterior. El contador no está defectuoso; responde fielmente a los objetos del componente. Hace falta obtener otra respuesta para actualizar esa representación.

En una aplicación con actualización automática podrían existir otros mecanismos, pero el laboratorio utiliza un botón explícito para mantener visible la relación entre acción, solicitud, respuesta y renderizado. Esta elección es pedagógica y suficiente para la consolidación del proyecto.

5.1.44. El éxito de integración se compone de hechos independientes

Una pantalla que abre demuestra que el host Blazor sirve la ruta. Un GET directo 200 con JSON demuestra que la API dispone del endpoint. Un listado visible demuestra que el componente obtuvo y representó datos. Un POST 201 con DTO demuestra una creación en el servidor según su contrato. Una recarga posterior demuestra que el componente puede obtener otra instantánea. Un error recuperable demuestra que la interfaz maneja una incidencia externa sin quedar bloqueada.

Ninguna observación por sí sola sustituye las demás. Por eso P5.1 solicita tanto operaciones desde el navegador como peticiones externas y variantes controladas de errores. El alumno no necesita añadir una arquitectura empresarial para entenderlo; necesita poder seguir con precisión cada frontera.

Esta capacidad completa el recorrido de Blazor B en el curso. Los puntos siguientes reutilizan la aplicación MVC A y se centran en otras fronteras: fuente y publicación, proceso y puerto, IIS y módulo de hospedaje, imagen y contenedor.

5.2. Ejecutar, compilar y publicar una aplicación ASP.NET Core MVC

5.2.1. ¿Por qué compilar no equivale a publicar?

Durante los módulos anteriores utilizamos Visual Studio o `dotnet run` para ejecutar las aplicaciones desde sus proyectos. Eso permite modificar el código, detenerlo con facilidad y observar la consola. Pero un directorio de proyecto incluye archivos de desarrollo, dependencias y un contexto que no se debería presuponer en el equipo que hospedará una aplicación.

Compilar transforma código fuente y recursos en artefactos utilizables. **Ejecutar** inicia un proceso. **Publicar** prepara los archivos que necesitaremos para llevar la aplicación a otro directorio o equipo compatible. No son sinónimos: un programa puede compilar correctamente y, sin embargo, fallar durante su arranque porque carece de una configuración o de una base a la que acceder.

Observemos tres instrucciones distintas:

```
dotnet build -c Release
dotnet run --no-launch-profile --urls http://localhost:5091
dotnet publish -c Release -o .\publicado
```

La primera crea ensamblados en un directorio de salida de compilación. La segunda inicia el host ASP.NET Core desde el proyecto (aunque internamente compile si se necesita). La tercera prepara una carpeta con ensamblados, recursos, dependencias, configuración y archivos necesarios para el despliegue. No debemos confundir la carpeta `bin/Release/net8.0/` con la salida controlada de `dotnet publish`: tienen usos y garantías distintas.

El modificador `-c Release` selecciona una configuración de compilación. No desactiva por sí mismo validaciones del servidor, no hace segura la aplicación y tampoco impide que su entorno de ejecución se configure como `Development`; son conceptos diferentes. El parámetro `-o` establece una salida explícita que facilita inspeccionar exactamente los archivos publicados.

5.2.2. Qué contiene la carpeta publicada

Un proyecto web típico de .NET 8 con despliegue dependiente del framework genera, entre otros, los siguientes elementos (los nombres exactos dependen del proyecto y de sus dependencias):

Elemento	Función habitual
`CursoWeb.Mvc.dll`	Ensamblado de la aplicación
`CursoWeb.Mvc.deps.json`	Dependencias de ejecución
`CursoWeb.Mvc.runtimeconfig.json`	Información del runtime requerido
`appsettings.json`	Configuración distribuida con la aplicación
`web.config`	Configuración del módulo ASP.NET Core para despliegue IIS cuando corresponde
`wwwroot/`	Archivos estáticos que la aplicación sirve

Un archivo DLL no es una «página web». Es código que el host ejecuta y que, junto con el routing y los controladores, produce respuestas HTTP. Una vista Razor no tiene por qué aparecer como archivo `.cshtml` independiente después de la publicación: dependiendo de la configuración, se compila como parte de los ensamblados. Por eso la comprobación correcta no consiste únicamente en contar archivos, sino en iniciar la salida publicada y solicitar páginas reales.

5.2.3. Dos modos de publicación y su diferencia

Para un despliegue **dependiente del framework**, el equipo de destino debe disponer de un runtime ASP.NET Core compatible. Podemos ejecutar la salida con:

```
dotnet .\publicado\CursoWeb.Mvc.dll
```

En un despliegue **autocontenido**, la publicación incorpora los componentes del runtime para una plataforma concreta, con mayor tamaño y opciones diferentes de ejecución. Es un concepto útil, pero no hace falta utilizarlo para entender el laboratorio. El itinerario emplea la publicación dependiente del framework en Windows y las imágenes oficiales .NET 8 en Docker. No mezclaremos distintas modalidades en un mismo experimento sin explicar sus requisitos.

La arquitectura y el sistema operativo también importan. Los recursos de publicación para Windows y Linux no pueden considerarse idénticos en todas las dependencias nativas, aunque el mismo código C# pueda compilar para ambas plataformas. En nuestros ejemplos no introduciremos paquetes nativos adicionales.

5.2.4. Configuración separada del programa

La aplicación MVC A incluye una cadena de conexión de SQL Server Express LocalDB. Una publicación copia la configuración correspondiente, pero no copia una instancia SQL Server ni los registros de la base. Si se ejecuta en otro equipo Windows sin LocalDB, las páginas dependientes del CRUD pueden fallar durante una consulta o incluso al arrancar si la aplicación inicializa la base en el inicio.

Consideremos una configuración didáctica alternativa:

```
{
  "Despliegue": {
    "SinLocalDb": true
  }
}
```

El significado no es «usar otra base», sino **ejecutar solo las rutas que no la necesitan** en un ensayo de despliegue. La aplicación de prácticas conserva la configuración original de SQL Server para el CRUD de Windows; el modo especial se selecciona explícitamente cuando se necesita comprobar el hospedaje sin disponer de LocalDB.

Las variables de entorno de .NET pueden sobrescribir claves de configuración mediante el separador `__`:

```
$env:Despliegue__SinLocalDb = "true"
```

Es una técnica didáctica válida para aislar una configuración del proceso. Debemos cerrar o restablecer esa variable antes de probar el CRUD persistente en Windows; si permaneciera activada, el experimento dejaría de estar midiendo el mismo comportamiento.

5.2.5. Compilación correcta frente a aplicación funcional

Un caso habitual: `dotnet publish` termina con código de salida cero y después `dotnet .\publicado\CursoWeb.Mvc.dll` falla. No implica que la compilación hubiera mentido; significa que la comprobación de compilación no cubría una condición de arranque. Puede faltar una cadena de conexión, estar ocupado un puerto o no estar instalado el runtime.

Conviene trabajar con evidencias diferenciadas:

Comprobación	Qué demuestra	Qué no demuestra
`dotnet build` finaliza correctamente	El proyecto compila con las dependencias disponibles	Que pueda llegar a LocalDB
`dotnet publish` genera salida	Existen artefactos de publicación	Que el sitio responda HTTP
Iniciar la DLL publicada	El proceso arranca con esa configuración	Que todas las rutas funcionen
Solicitar `/` y `/Productos/Estado`	Hay respuesta desde rutas sin base	Que las operaciones CRUD estén disponibles
Crear un producto en Windows/LocalDB	El recorrido MVC → EF Core → SQL funciona	Que el mismo CRUD funcione en Docker Linux

5.2.6. Síntesis

La transición de ejecutar a publicar se entiende cuando el alumno reconoce qué archivos se han producido, cuáles son las dependencias del destino y qué evidencia requiere un programa que arranca fuera de Visual Studio. Esos conocimientos son prerrequisitos de Kestrel, IIS y Docker.

5.2.7. Qué cambia realmente cuando dejamos de ejecutar desde el proyecto

Mientras desarrollamos con Visual Studio, trabajamos con un directorio que contiene el archivo `.csproj`, código C#, vistas, recursos y configuraciones. La herramienta puede compilarlo, iniciar el proceso y ofrecer una experiencia de depuración. Esa comodidad no debe confundirse con los archivos que hay que trasladar al destino.

`dotnet build` permite comprobar que el código y las referencias forman una aplicación compilable. `dotnet publish` prepara una salida adecuada para ejecutar el programa con unas opciones de despliegue determinadas. `dotnet run` inicia la aplicación desde su proyecto, compilando si corresponde. Una publicación no inicia un servidor por sí misma y una compilación correcta no acredita una conexión a SQL Server.

En una publicación dependiente del framework de ASP.NET Core 8, el directorio de destino debe disponer de un runtime compatible. El ensamblado de MVC A está en `CursoWeb.Mvc.dll`, pero no es suficiente por sí solo. Necesita los archivos de dependencias y configuración correspondientes, las bibliotecas pertinentes y, en el caso de una aplicación web, los recursos estáticos que debe servir.

El criterio correcto para observar la transición es identificar tres preguntas independientes: **¿se ha construido el programa?**, **¿se han preparado los archivos para el destino?** y **¿hay un proceso atendiendo peticiones reales con esos archivos?**. Una respuesta afirmativa a la primera no determina las otras dos.

5.2.8. Leer la carpeta publicada como un conjunto de responsabilidades

`CursoWeb.Mvc.runtimeconfig.json` aporta información sobre el entorno de ejecución requerido. Si el equipo no tiene instalado el runtime adecuado, el proceso puede fallar antes de atender ninguna ruta MVC. `CursoWeb.Mvc.deps.json` participa en la resolución de dependencias; no es una tabla de rutas que debamos editar para hacer aparecer una vista. La aplicación sigue ejecutando código compilado.

`wwwroot` contiene archivos estáticos; el programa debe tener configurado su servicio. Si `/` responde HTML y `/css/site.css` devuelve 404, el servidor y la ruta inicial pueden funcionar mientras falta un recurso. Por eso la simple visita a la página inicial es una prueba incompleta. Los archivos de configuración, a su vez, pueden contener una cadena de conexión que apunta a un recurso no disponible. Copiar el archivo no crea la base de datos.

Cuando se publica para IIS puede aparecer `web.config`, que describe cómo debe iniciar la aplicación el módulo de hospedaje ASP.NET Core. Ese archivo es significativo para IIS, pero su existencia no instala IIS ni el Hosting Bundle. Más adelante veremos que el mismo programa puede recibir solicitudes mediante otros mecanismos sin cambiar sus controladores.

5.2.9. Directorio actual y directorio publicado no son necesariamente iguales

Una ruta relativa se interpreta desde un contexto. El comando `dotnet publish .\CursoWeb.Mvc.csproj` presupone que el proyecto se encuentra en la carpeta actual. Si nos situamos en la raíz del repositorio, habrá que utilizar una ruta distinta, y si abrimos la carpeta de salida publicada tendremos que ejecutar la DLL desde allí o indicar su ruta completa.

Esta diferencia se vuelve visible cuando aparecen errores de «archivo no encontrado». No significa que el programa C# esté mal diseñado. Puede tratarse de una orden ejecutada desde otra carpeta.

La respuesta técnica es comprobar el directorio actual, el archivo de proyecto y la ubicación de salida, no copiar código al azar a distintos directorios.

Además, no hay que suponer que editar una vista en el árbol de código modifica una publicación ya construida. Una distribución normalmente conserva la versión de archivos y ensamblados producida cuando se publicó. Para incorporar cambios posteriores se vuelve a publicar y se sustituye la salida que ejecuta el destino.

5.2.10. Configuración de compilación y entorno de ejecución son conceptos distintos

La opción `-c Release` determina una configuración de construcción. No significa que el entorno ASP.NET Core vaya a llamarse necesariamente `Production`. Tampoco instala certificados HTTPS ni añade autorización a los controladores. Es un error pedagógico presentar Release como sinónimo de seguridad.

La aplicación puede leer una configuración general y valores específicos del entorno; las variables de entorno suelen utilizar `__` para representar claves jerárquicas como `Despliegue:SinLocalDb`. Una ejecución puede, por tanto, comportarse de manera distinta aunque se inicie exactamente el mismo ensamblado:

```
$env:Despliegue__SinLocalDb = "true"
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

El ejemplo presupone que estamos dentro del directorio publicado. Con esta opción la aplicación del curso realiza únicamente el recorrido sin base previsto para demostrar hospedaje; no convierte automáticamente la cadena LocalDB en una conexión válida. Restablecer el valor es imprescindible cuando volvemos a trabajar con el CRUD persistente de M02.

5.2.11. Por qué LocalDB puede impedir el arranque antes de que exista una página

El `Program.cs` de MVC A registra un contexto EF Core con `UseSqlServer`. Registrar el contexto no implica que todas las llamadas a SQL Server hayan tenido lugar ya. Sin embargo, el ejemplo del curso también realiza una inicialización elemental de base al inicio bajo la configuración normal. Si esa operación falla, el proceso puede terminar sin haber empezado a escuchar HTTP.

Imaginemos que alguien copia el directorio publicado a un equipo que no dispone de SQL Server Express LocalDB. Ve la DLL y un archivo de configuración correcto, pero la aplicación no arranca. Concluir que Kestrel «no funciona» sería precipitado: la causa inmediata puede estar en una dependencia de datos anterior a la apertura del puerto.

En el modo de ensayo de despliegue, el programa consulta `Despliegue:SinLocalDb`. Cuando vale `true`, omite esa inicialización, pero no pretende habilitar el CRUD con otro proveedor. Para evitar un resultado engañoso, las rutas de productos que requerirían SQL devuelven explícitamente un **503**. Así queda separada la disponibilidad del servidor web de la disponibilidad de la funcionalidad de datos.

5.2.12. Historia de una publicación que funciona parcialmente

Un alumno construye MVC A y publica a carpeta. La orden termina correctamente, inicia la aplicación y abre `/` con respuesta 200. Sin embargo, cuando intenta acceder a `/Productos`, recibe 503. ¿Qué sabemos realmente?

Sabemos que existe una publicación ejecutable, un proceso que responde, una ruta MVC disponible y una ruta que declara no estar disponible. Todavía no sabemos si está instalada LocalDB ni cuál fue la configuración de arranque. Debemos mirar el valor del indicador de despliegue y el contexto del proceso antes de modificar las vistas o rehacer la base.

En otra máquina puede suceder algo diferente: `/` funciona, pero el CSS falta. Aquí el problema afecta a recursos estáticos, no necesariamente a SQL Server. Y en un tercer equipo la aplicación ni siquiera llega a abrir un puerto porque el runtime esperado no está instalado. Las tres situaciones se describen coloquialmente como «la publicación falla», pero requieren diagnósticos diferentes.

Esa es la finalidad de este apartado: dar significado técnico a la palabra publicar. Los comandos son relevantes porque producen artefactos concretos, pero la capacidad que debe adquirir el alumno consiste en explicar **qué ejecución se ha preparado, qué dependencias conserva y qué evidencias demuestran sus límites**.

5.2.13. Dos publicaciones del mismo proyecto pueden mostrar vistas diferentes

Consideremos la aplicación MVC A. Tiene una vista `Views/Home/Index.cshtml` que muestra un texto sencillo. Un alumno publica el proyecto a la carpeta A. Después modifica el texto de la vista y publica a la carpeta B. Ambas publicaciones contienen artefactos de **la misma aplicación**, pero se construyeron en momentos diferentes.

Si iniciamos la DLL de A en el puerto 5091 y la DLL de B en el 5094, cada proceso sirve el contenido correspondiente a los archivos con los que se inició. El hecho de haber editado después el proyecto fuente no obliga a una publicación anterior a actualizar sus vistas compiladas.

Este ejemplo elimina una intuición incorrecta: «tengo el proyecto abierto con el nuevo texto; luego todos los servidores que inicié antes ya utilizan esa versión». La publicación es una operación que prepara artefactos en un instante concreto. Ejecutar una publicación significa utilizar esos artefactos, no volver continuamente al editor para leer las últimas modificaciones.

Recorrido conceptual:

```
Proyecto MVC — Vista original
       |
       +--- publicar en A ------> A / ensamblados / recursos
       |                                |
       |                                +--> Kestrel en 5091
       |
Modificar la vista en el proyecto
       |
       +--- publicar en B ------> B / ensamblados / recursos
                                        |
                                        +--> Kestrel en 5094
```

Ambos procesos pueden devolver HTTP 200 para `/`, pero ese código no garantiza que la representación HTML sea idéntica. Para comprobar el cambio hay que buscar el contenido esperado en cada respuesta, no limitarse al estado HTTP.

Esta comparación enseña a localizar un problema frecuente en despliegues: el administrador copió los archivos nuevos a una carpeta, pero el servicio está apuntando a otra. Antes de cambiar código de negocio, debe verificarse qué directorio utiliza el proceso realmente.

5.2.14. El cuerpo de una respuesta confirma qué versión está en uso

Cuando examinamos un despliegue, el estado 200 responde a la pregunta «¿se completó satisfactoriamente esta solicitud HTTP?». No responde necesariamente a «¿es esta la versión del programa que pretendía desplegar?». Una aplicación antigua puede servir perfectamente HTTP 200.

Por ejemplo, si la segunda publicación añade:

```
<p id="marca-publicacion-m05">
    Esta versión se publicó después de modificar la vista MVC.
</p>
```

el contenido de la respuesta nos permite distinguir las dos salidas. Ese marcador no cambia el controlador ni el enrutamiento; solo cambia el HTML que la vista produce. Es un experimento especialmente útil porque evita introducir una nueva dependencia o una nueva aplicación para verificar el efecto de publicar.

El alumno debe poder explicar tres hechos separados: la publicación A fue creada antes del cambio, la publicación B se creó después y los procesos apuntan a carpetas distintas. Si ambos muestran la misma página, revisar primero la ruta de publicación y el proceso activo suele ser más razonable que alterar el modelo de productos.

5.2.15. La ruta relativa puede engañar incluso cuando el comando es correcto

La opción `-o` de `dotnet publish` recibe una ruta de salida. Una ruta relativa se interpreta respecto al directorio desde el que se ejecuta el comando. Si el alumno cree que está en la raíz del repositorio, pero se encuentra dentro del directorio del proyecto, la misma ruta textual puede producir una carpeta distinta.

Por eso la práctica define rutas desde una raíz conocida y utiliza una carpeta `.work` ignorada por Git. Separar esos resultados del código del proyecto evita mezclar archivos compilados con fuentes o incorporarlos por accidente a un commit.

Ejemplo de duda frecuente: «He publicado, pero no encuentro la DLL en la carpeta indicada». La primera comprobación no es reinstalar .NET ni regenerar el proyecto. Es consultar el directorio actual, la salida de `dotnet publish` y la ubicación efectiva donde se escribieron los archivos.

En una práctica bien preparada el alumno aprende a leer el mensaje de publicación, no solo a recordar qué parámetro escribió. La salida de una orden y la estructura del sistema de archivos son evidencias complementarias.

5.2.16. Los recursos estáticos y las vistas siguen caminos distintos

Una vista MVC es código de presentación Razor que participa en la producción dinámica de HTML. Una hoja CSS situada en `wwwroot`, en cambio, es un archivo estático que el middleware puede entregar como recurso. Aunque ambos afecten a cómo se ve una página, sus mecanismos no son idénticos.

La publicación prepara recursos estáticos y ensamblados. Un fallo de una hoja de estilos puede hacer que la página tenga aspecto incorrecto aunque la acción Home/Index esté funcionando. Por el contrario, un fallo al ejecutar una acción MVC puede impedir producir HTML aunque `wwwroot` siga estando disponible.

Al comprobar la publicación conviene solicitar ambos tipos de contenido: la raíz `/`, que pasa por MVC, y `/css/site.css`, que corresponde a un recurso estático. La combinación permite descartar algunas hipótesis sin modificar el programa.

Una respuesta 404 para CSS obliga a preguntar si el archivo se publicó en la ruta esperada, si la URL solicitada corresponde a su ubicación y si está configurado el middleware de archivos estáticos. Un 404 para una ruta MVC puede exigir revisar el enrutamiento o el nombre de la acción. **El mismo código HTTP no implica la misma causa interna.**

5.2.17. La publicación dependiente del framework conserva una obligación en el destino

En el itinerario se utiliza un despliegue dependiente del framework. Los archivos publicados no contienen necesariamente todos los componentes de runtime .NET que requiere la aplicación. El destino debe contar con una versión compatible del runtime ASP.NET Core 8.

Esa decisión reduce el tamaño de los artefactos distribuidos y encaja con equipos de laboratorio que tienen .NET 8 instalado. No significa que copiar la DLL a cualquier equipo Windows sea suficiente para ejecutarla. Una máquina sin el runtime requerido puede mostrar un mensaje de ejecución que indique el framework ausente.

El despliegue autocontenido es otra modalidad posible, con tamaño y requisitos diferentes. Se menciona para entender la distinción, pero no se añade como un segundo itinerario obligatorio. Mantener un único modo de publicación permite comparar Kestrel, IIS y Docker sin introducir variables de plataforma innecesarias.

En Docker, la imagen final basada en `mcr.microsoft.com/dotnet/aspnet:8.0` proporciona el runtime del contenedor. El proceso que se inicia allí sigue siendo la publicación de MVC A, no una aplicación transformada en otro framework.

5.2.18. La existencia de una configuración no garantiza la disponibilidad de sus recursos

El archivo `appsettings.json` puede contener una cadena `ConnectionStrings:Productos`. Esa cadena es una instrucción sobre cómo localizar una base, pero no instala SQL Server Express LocalDB. Tampoco asegura que la identidad del proceso pueda abrir la instancia o que la base de datos exista.

Este punto es fundamental para interpretar la publicación del CRUD. En Windows, bajo el usuario que ha preparado LocalDB, puede ser posible crear y consultar productos. Bajo la identidad de un Application Pool de IIS, la misma cadena puede no encontrar una instancia accesible. Dentro de un contenedor Linux, la instancia LocalDB de Windows no está presente de forma ordinaria.

La aplicación no puede inventar una base disponible por el simple hecho de ejecutarse correctamente. En M05 se ha añadido un modo explícito que aísla las rutas independientes de SQL y rechaza con 503 las dependientes. No se ha cambiado la tecnología de persistencia.

Conclusión: dos publicaciones con el mismo código pueden ofrecer capacidades distintas porque se ejecutan con diferentes identidades y dependencias. Una clase C# no cambia, pero el contexto en que opera sí.

5.2.19. Una comprobación completa necesita al menos cuatro observaciones

Para evaluar una publicación sencilla no basta con una captura del explorador de archivos. Conviene reunir cuatro observaciones en secuencia.

Primero, el resultado de `dotnet publish`: debemos saber que la operación terminó y dónde dejó la salida. Segundo, la presencia del ensamblado y los archivos necesarios en esa carpeta. Tercero, el arranque de un proceso que utiliza esa salida, distinguiendo su dirección de escucha. Cuarto, una solicitud HTTP concreta cuyo resultado podamos interpretar.

Podemos ampliar la última observación a diferentes recursos: HTML MVC, estado del controlador y CSS. La prueba de operaciones CRUD con LocalDB corresponde a otro escenario en el que la dependencia de datos esté disponible.

Las cuatro observaciones responden a preguntas diferentes; juntas construyen una explicación sólida de lo que significa publicar. El alumno no necesita ejecutar un pipeline empresarial ni implantar infraestructura avanzada para comprenderlas.

5.2.20. Caso completo: he publicado, pero sigo viendo la pantalla anterior

Un desarrollador modifica el texto de Home/Index, ejecuta `dotnet publish` y sigue viendo la página antigua. La primera reacción suele ser repetir el comando hasta que cambie. Una investigación más útil comienza localizando qué proceso escucha el puerto consultado.

Si el proceso inició la publicación A y el desarrollador acaba de publicar en B, no hay motivo para que el proceso A cambie automáticamente. Incluso si se vuelve a abrir la página, seguirá solicitando a A. Necesitamos comprobar la carpeta que utiliza el proceso, detenerlo si corresponde e iniciar la salida nueva.

Si el mecanismo de hospedaje fuera IIS, habría que revisar su ruta física. Si fuera Docker, habría que distinguir la imagen anterior de la nueva y comprobar qué imagen utiliza el contenedor. La causa general es la misma: **editar o construir no equivale automáticamente a sustituir la instancia en ejecución**.

Este principio conecta los tres mecanismos de publicación del módulo sin convertirlos en tecnologías aisladas. En cada uno existe alguna forma de seleccionar los artefactos que ejecuta el host. Reconocer esa selección permite explicar muchos errores antes de tocar el código fuente.

5.2.21. Publicar una aplicación no significa trasladar su entorno de desarrollo

Durante los días 1 y 2, MVC A fue evolucionando dentro de una solución de Visual Studio. El editor conocía el archivo de proyecto, ofrecía las vistas Razor y permitía ejecutar y depurar la aplicación. Ese entorno es una ayuda para construir el programa, no un requisito que deba copiarse íntegramente al servidor de destino.

Una publicación dependiente del framework contiene artefactos del programa y presupone que el destino dispone de las condiciones de ejecución indicadas. Podemos imaginar el equipo de desarrollo con Visual Studio y LocalDB instalados y otro equipo con el runtime ASP.NET Core pero sin LocalDB. Copiar los archivos publicados permite intentar iniciar el host, pero la operación de inicialización SQL del `Program.cs` normal podría fallar.

El resultado nos enseña a diferenciar **dependencias de construcción**, **dependencias de ejecución** y **dependencias funcionales**. Visual Studio y el SDK pertenecen a la construcción. El runtime y los ensamblados pertenecen a la ejecución. SQL Server Express LocalDB pertenece a la funcionalidad que realmente necesita persistencia. No todas las rutas del mismo sitio tienen por qué utilizar esa dependencia funcional.

Por eso una publicación correcta no puede prometer más de lo que sus condiciones permiten. Al trasladarla a IIS, nos interesará comprobar qué identidad tiene el proceso y qué módulos de hospedaje existen. Al trasladarla al contenedor Linux, tendremos un runtime ASP.NET Core dentro de la imagen, pero no una instancia LocalDB de Windows.

5.2.22. La diferencia entre un directorio preparado y un servidor escuchando

La operación `dotnet publish` escribe archivos. Puede finalizar satisfactoriamente mientras no exista ningún servidor HTTP atendiendo. El explorador de Windows puede mostrar `CursoWeb.Mvc.dll`, `CursoWeb.Mvc.deps.json` y otros archivos; eso no significa que el navegador tenga una URL disponible.

Un servidor aparece cuando se ejecuta el programa en un proceso con configuración válida, se preparan sus dependencias y se abre la escucha. La dirección utilizada depende del mecanismo de hospedaje. En Kestrel podemos indicar una URL local; en IIS la dirección visible se configura mediante bindings; en Docker la dirección del host requiere publicar el puerto del contenedor.

Estas diferencias explican por qué la práctica avanza primero desde `dotnet build`, después hacia `dotnet publish`, luego a un proceso Kestrel y finalmente a otros mecanismos. No se debe comenzar creando un sitio IIS antes de saber qué carpeta publicada se va a utilizar.

Un síntoma didáctico habitual es que el alumno declara «ya está publicada» tras comprobar únicamente la salida de compilación. Una formulación rigurosa sería: «se ha generado el directorio de publicación»; solo después de una petición real podrá añadir «el host está atendiendo esta ruta».

5.2.23. Comprobar el mismo comportamiento después de dos cambios de contexto

La primera ejecución de MVC A sucede desde el proyecto, con `dotnet run`. La segunda sucede desde la salida publicada, con `dotnet CursoWeb.Mvc.dll`. Queremos reconocer la continuidad: el controlador `HomeController`, la vista Razor y los recursos siguen perteneciendo al mismo proyecto.

No se necesita modificar el código para crear un controlador llamado `HomePublicadoController`. Tampoco hace falta que cada modo de ejecución tenga su propia carpeta `Views` dentro de la solución. La publicación transforma y prepara los artefactos necesarios para servir la aplicación existente.

Una prueba significativa consiste en abrir `/` desde el proceso inicial, detenerlo, iniciar la publicación en el mismo puerto y volver a abrir `/`. Si el contenido corresponde a la misma versión, el resultado visible puede ser idéntico, pero el contexto de ejecución habrá cambiado. La evidencia adicional es el comando de inicio y el directorio del que procede la DLL.

Si se modificó una vista entre las dos publicaciones, podremos observar un contenido diferente. Eso no contradice la continuidad del proyecto: confirma que se ha ejecutado una versión distinta de sus artefactos.

5.2.24. El uso de una carpeta de trabajo temporal facilita la reproducibilidad

El repositorio debe conservar los proyectos fuente y los manuales, no múltiples directorios de salida improvisados. Por eso las instrucciones docentes utilizan `.work/M05` para alojar publicaciones de ensayo. La carpeta de trabajo permite repetir ejercicios, comparar dos versiones y limpiar sus resultados sin alterar la implementación.

La práctica de dos publicaciones se apoya en esta idea: publica antes de cambiar la vista en una carpeta A, y después en una carpeta B. Al mantener esos destinos separados, puede demostrarse que el proceso A no cambia automáticamente cuando existe una publicación B.

No debemos confundir el directorio `.work` con una solución o un módulo versionado. Tampoco es un almacén de resultados que el alumno tenga que entregar al repositorio. Es un espacio temporal para practicar el recorrido publicación→ejecución sin sobrescribir el código ni introducir archivos generados junto a las vistas de origen.

5.2.25. Si una respuesta es antigua, identificar los artefactos que utiliza el proceso

Imagina que el alumno actualiza la portada de MVC y vuelve a publicar. Sin embargo, el navegador sigue mostrando el texto anterior. Antes de modificar el controlador, debemos comprobar tres elementos:

Primero, si la publicación nueva se produjo en la carpeta que el alumno esperaba. Una ruta relativa incorrecta puede haber escrito archivos en otro directorio.

Segundo, si el proceso en ejecución fue iniciado desde la publicación nueva o desde una anterior. Un proceso Kestrel que seguía escuchando en el puerto conserva su instancia y sus artefactos.

Tercero, si el navegador está consultando la dirección correcta. Dos procesos pueden estar escuchando en puertos diferentes y representar versiones distintas sin que exista ningún error.

Esta investigación no requiere un sistema de despliegue avanzado. Solo necesita relacionar cambios de fuente, salida publicada, proceso y solicitud. Más adelante, en IIS, será la ruta física del sitio la que indique la publicación usada. En Docker será la imagen con la que se creó el contenedor.

5.2.26. Qué información aporta una ruta que no depende de SQL Server

La acción `/Productos/Estado` devuelve un texto breve y no necesita consultar la tabla de productos. Gracias a esa independencia, sirve para reconocer si el host ASP.NET Core, su routing y el controlador pueden responder aun cuando el escenario de despliegue no dispone de LocalDB.

Una respuesta 200 en esa ruta no debe interpretarse como demostración de que los productos se pueden crear. Tampoco una respuesta 503 en `/Productos` significa necesariamente que el servidor web no está ejecutándose. El conjunto de observaciones distingue la capa web de una dependencia que falta deliberadamente.

Esa separación es especialmente útil en una jornada introductoria de despliegue. No queremos introducir configuración empresarial de SQL Server accesible desde Linux únicamente para abrir la portada de MVC. Queremos que el alumno sepa decir: «el mecanismo de hospedaje funciona; la función de catálogo persistente no forma parte de este ensayo».

Si se quiere comprobar el CRUD, hay que volver al contexto Windows con la instancia LocalDB preparada, desactivar la opción sin base y ejecutar operaciones reales. El curso nunca sustituye la tecnología de base de datos por SQLite para evitar ese requisito.

5.2.27. Publicación, entorno y seguridad: no extraer conclusiones indebidas

Un directorio publicado con `-c Release` puede ser técnicamente adecuado para ejecutar la aplicación y, al mismo tiempo, carecer de una configuración segura para Internet. La optimización de compilación y la protección frente a accesos externos pertenecen a problemas distintos.

De la misma forma, elegir `ASPNETCORE_ENVIRONMENT=Production` no activa automáticamente certificados, autorización o una política de seguridad. Los valores del entorno determinan qué configuración se carga y pueden modificar comportamientos concretos, pero no sustituyen decisiones explícitas de administración y diseño.

La práctica de IIS usa HTTP local y un puerto de laboratorio. Docker publica un puerto del host para hacer visible el contenedor en el mismo equipo. Ninguna de esas decisiones debe copiarse sin revisión a un despliegue público. El temario original delimitó deliberadamente una introducción sencilla, no un manual para operar servicios expuestos a Internet.

El aprendizaje esperado es poder señalar las condiciones que cambian y reconocer qué garantías no se han ensayado. Una demostración limitada y correctamente explicada es preferible a una afirmación general de que el programa está «listo para producción» por el mero hecho de responder en un navegador.

5.2.28. Leer una publicación como evidencia acumulativa

Podemos establecer una cadena de resultados concretos sin crear un sistema de auditoría dentro del material del alumno. Después de `dotnet build` sabemos que la compilación ha terminado. Después de `dotnet publish` sabemos que se ha preparado una salida. Después de iniciar la DLL sabemos que el proceso ha llegado a ejecutarse. Después de GET a `/` sabemos que una ruta ha respondido. Después de GET a `/css/site.css` sabemos que se ha entregado un recurso estático. Después de una operación real de CRUD con SQL Server sabemos que esa operación y su dependencia han respondido.

La secuencia evita una inferencia equivocada: que el paso anterior acredita automáticamente todos los siguientes. Una publicación puede compilar y carecer del runtime en destino; un host puede responder la portada y carecer de base de datos.

Esta forma de argumentar es una competencia profesional transferible. El alumno debería poder explicar qué observación aporta cada acción y cuál es el siguiente hecho que necesita comprobar. No debe limitarse a recitar comandos; debe comprender qué estado han producido.

5.2.29. Comparación de resultados con y sin LocalDB

Condición de ejecución	Inicialización de SQL	Página inicial	Ruta Estado	Rutas CRUD
Windows con LocalDB disponible y modo normal	Se intenta y puede completarse	Disponible si el host arranca	Disponible	Comprobables con EF Core
Windows sin LocalDB y modo normal	Puede fallar durante el arranque	No garantizada	No garantizada	No garantizadas

Condición de ejecución	Inicialización de SQL	Página inicial	Ruta Estado	Rutas CRUD
Windows en modo sin LocalDB	Se omite por diseño	Disponible si el host arranca	Disponible	503 explícito
Docker Linux en modo sin LocalDB	Se omite por diseño	Disponible si el host arranca	Disponible	503 explícito

La tabla no sustituye las pruebas. Explica qué resultado sería coherente con cada condición. Si bajo el modo sin base la ruta de productos devolviera 200 con un catálogo fijo improvisado, la demostración sería engañosa: parecería que el CRUD funciona sin la infraestructura acordada. En cambio, el 503 mantiene visible el límite.

5.2.30. Cierre: de la carpeta publicada al primer servidor

El alumno está preparado para pasar a Kestrel cuando puede localizar el archivo de proyecto, producir una publicación, interpretar los archivos principales, elegir un directorio de ejecución y distinguir la disponibilidad de una página de la persistencia de productos.

El siguiente punto no vuelve a enseñar MVC desde cero. Reutiliza el mismo proyecto y responde a otra pregunta: ¿qué proceso acepta solicitudes HTTP cuando ejecutamos la publicación y cómo interpretamos la relación entre puerto, ruta y resultado? La continuidad es deliberada. El nuevo aprendizaje se añade al recorrido previo, no lo sustituye.

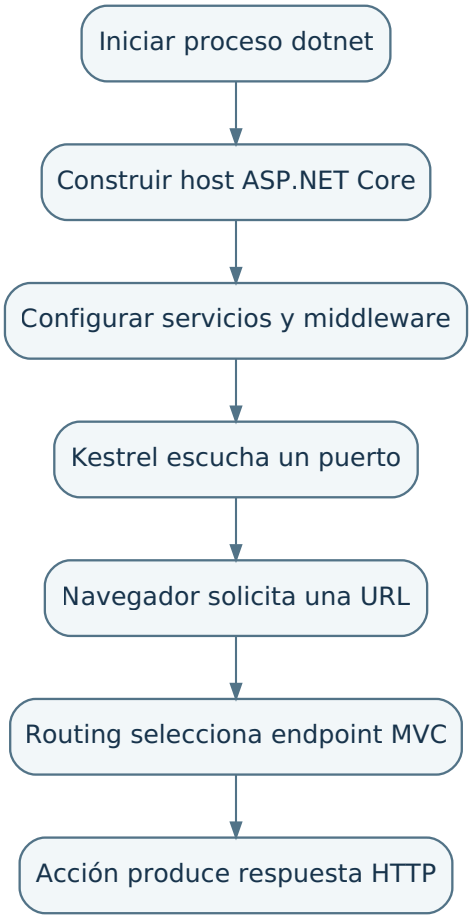
5.3. Kestrel: proceso, dirección y respuesta HTTP

5.3.1. Qué es Kestrel y qué hace

Kestrel es el servidor web multiplataforma utilizado por ASP.NET Core. Cuando iniciamos una aplicación con `dotnet run` o ejecutamos una DLL publicada como aplicación web, un host ASP.NET Core puede abrir un socket y escuchar peticiones HTTP mediante Kestrel. El navegador no ejecuta C#: envía solicitudes a una dirección que el proceso atiende. Un controlador MVC decide qué hacer con cada ruta y devuelve la respuesta correspondiente.

En nuestro laboratorio bastará con ejecutar una publicación desde una carpeta concreta, establecer una dirección local y observar la respuesta. No necesitamos configurar balanceadores, proxies corporativos, certificados de producción ni límites avanzados.

El recorrido conceptual puede describirse así:



El dibujo diferencia el arranque (una vez por proceso) de las peticiones (tantas como el servidor pueda atender). No debemos atribuir la creación de un nuevo proceso `dotnet` a cada petición.

5.3.2. Dirección de escucha, host y puerto

Supongamos esta orden:

```
dotnet .\CursoWeb.Mvc.dll --urls http://localhost:5091
```

`localhost` indica que se pretende escuchar en la máquina local. `5091` es el puerto elegido para el experimento; no es un número obligatorio de ASP.NET Core. En el navegador usaremos `http://localhost:5091/`. Si el comando muestra una dirección distinta porque otra configuración tiene mayor prioridad, debemos atender a la URL efectiva publicada por el host.

No mezclemos sin explicación tres direcciones diferentes: la de la aplicación MVC, la de Blazor y la de la API de productos. Durante 5.3 usamos solamente MVC. El proceso publica recursos HTML; no necesita que funcione la API de productos del otro laboratorio.

5.3.3. ¿Qué sucede si el puerto está ocupado?

Dos procesos no suelen poder escuchar simultáneamente la misma combinación de dirección y puerto. Si la aplicación MVC sigue ejecutándose desde Visual Studio y tratamos de arrancar una segunda copia de la publicación en ese puerto, el segundo proceso puede terminar con error de enlace a la dirección. No es un problema del código de una vista ni de Entity Framework Core.

La estrategia de diagnóstico consiste en comprobar si hay otro proceso escuchando y elegir: detenerlo o utilizar otro puerto.

```
Get-NetTCPConnection -LocalPort 5091 -ErrorAction SilentlyContinue
```

La orden pertenece a PowerShell en Windows y muestra conexiones asociadas al puerto cuando la plataforma la admite. Puede requerirse interpretar estados y procesos. Alternativamente cambiamos temporalmente la prueba a un puerto libre, por ejemplo 5094, y confirmamos qué URL anunció Kestrel.

5.3.4. Una respuesta de una ruta no acredita todas las rutas

Si `/` funciona y `/Productos/Estado` devuelve texto, hemos demostrado que Kestrel, el routing y esas acciones responden. Pero el CRUD de productos de M02 puede necesitar LocalDB. No debemos presentar «la página inicial abre» como si probara automáticamente las operaciones de crear, editar y eliminar.

El código HTTP que observamos se compone de una línea de estado, cabeceras y un cuerpo opcional. Podemos consultar desde el navegador o desde una terminal:

```
curl.exe -i http://localhost:5091/Productos/Estado
```

Una respuesta 200 con texto esperado es un resultado concreto; si se obtiene 503 para `/Productos` bajo el modo sin LocalDB, eso representa una limitación **deliberada y documentada**, no un CRUD funcional.

5.3.5. Detención del proceso y restauración del entorno

En una terminal interactiva, `Ctrl+C` suele solicitar el apagado del proceso. Es importante cerrar la aplicación que ya estaba escuchando antes de iniciar otra modalidad de hospedaje. Una terminación limpia facilita liberar el puerto y evita atribuir una respuesta al proceso equivocado.

Kestrel no es exclusivo de `dotnet run`. La publicación ASP.NET Core también puede atender peticiones mediante Kestrel, por lo que la diferencia formativa no reside en «servidor de desarrollo frente a servidor real», sino en los artefactos, su configuración y el modo en que se hospeda y administra el proceso.

5.3.6. Síntesis

El alumno podrá explicar qué proceso recibe las solicitudes, cómo conoce su puerto, qué sucede ante una colisión de dirección y cómo demostrar con una petición HTTP que una ruta concreta funciona.

5.3.7. El proceso que recibe la petición y el proceso que construyó el programa

La palabra «servidor» puede designar una máquina, un programa o una función de ASP.NET Core. En nuestro laboratorio necesitamos ser precisos: Kestrel es el servidor HTTP integrado en el host ASP.NET Core, mientras que el proceso `dotnet` es la ejecución concreta que lo inicia. La máquina Windows es el entorno que aloja ese proceso.

Cuando ejecutamos la DLL publicada, el código de `Program.cs` construye el host y configura servicios, middleware y rutas. Si no falla una dependencia previa, se inicia la escucha HTTP. Una vez iniciado, **cada petición no vuelve a ejecutar todo `Program.cs` desde cero**. El proceso conserva sus servicios según sus ámbitos de vida y atiende las solicitudes mediante el pipeline configurado.

Esa distinción ayuda a interpretar el ciclo de vida de los recursos. Una instancia de LocalDB puede sobrevivir al proceso web porque es un servicio de datos separado. Una colección estática mantenida dentro de una API de demostración no tiene esa misma garantía al reiniciar el proceso. Aunque ambos servidores se ejecuten con `dotnet`, sus datos no tienen el mismo ciclo de vida.

5.3.8. Escuchar, recibir, enrutar y responder son etapas diferentes

Que un programa escriba «Now listening on...» en la consola informa de su dirección de escucha. No demuestra que una ruta específica esté definida ni que la acción MVC pueda completarse. El servidor puede aceptar conexiones correctamente y devolver 404 porque no existe el recurso, o devolver 503 porque una función está deshabilitada para ese escenario.

Consideremos la secuencia completa de una solicitud `GET /Productos/Estado`:

1. Un cliente interpreta la dirección y decide el destino de la conexión.
2. El sistema operativo dirige el tráfico al proceso que escucha el puerto.
3. Kestrel interpreta la solicitud HTTP recibida.
4. El middleware ASP.NET Core participa en el procesamiento.
5. El enrutamiento encuentra un controlador y una acción compatibles.
6. La acción produce un resultado HTTP, con código de estado y cuerpo.
7. El servidor transmite esa respuesta al cliente, que la interpreta.

Si obtenemos un 404, las etapas iniciales pueden haber funcionado perfectamente. Si recibimos un texto esperado con 200, hemos demostrado mucho más que la existencia de un socket: también se ejecutó una acción determinada. Si la página inicial funciona pero el CSS no, el problema puede estar en la ruta de recursos estáticos y no en el arranque del host.

5.3.9. `localhost` no es una dirección universal de todas las máquinas

En este curso utilizamos intencionadamente `localhost` porque las demostraciones son locales. Es importante comprender que ese nombre se refiere al propio entorno desde el que se interpreta. Una URL `http://localhost:5091/` utilizada desde el navegador del alumno pretende conectar con el proceso que escucha en su máquina. La misma URL abierta en otro equipo apunta a **ese otro equipo**, no al primero.

Este detalle cobra importancia al introducir Docker. Dentro de un contenedor, `localhost` se refiere al espacio de red del propio contenedor. Para publicar una aplicación desde ese entorno utilizaremos una dirección de escucha apropiada en el contenedor y un mapeo de puertos hacia el host. No debemos trasladar de forma literal la noción de `localhost` entre procesos y entornos de red diferentes.

En Kestrel Windows del laboratorio, la exposición local es suficiente. No configuramos direcciones accesibles desde Internet ni certificados públicos; ese trabajo implicaría decisiones de seguridad e infraestructura que exceden M05.

5.3.10. La dirección solicitada puede no coincidir con la dirección efectiva

Una aplicación ASP.NET Core recibe opciones de escucha desde la línea de comandos y otras fuentes de configuración. Es posible que el alumno ejecute un comando con un puerto, pero otra configuración explícita del host lo modifique. Por ello no basta con recordar el número que se pretendía usar: debemos comprobar la dirección anunciada por el proceso y el resultado de una solicitud.

El procedimiento de diagnóstico no empieza cambiando puertos de forma aleatoria. Primero observamos qué URL utiliza realmente el proceso. Después comprobamos si hay otro servicio escuchando en el puerto previsto. Por último, si es necesario, elegimos un puerto distinto y repetimos una petición a una ruta conocida.

En Windows, este ejemplo de PowerShell permite observar los procesos asociados a un puerto sin matar ninguno:

```
Get-NetTCPConnection -LocalPort 5091 -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, State, OwningProcess
```

La ausencia de filas no prueba por sí sola una única causa; puede depender de permisos, de cómo escucha el proceso o de cuándo se realizó la comprobación. El comando es una evidencia complementaria a los mensajes del host y a una petición HTTP.

5.3.11. Una colisión de puerto no es una excepción de controlador

Supongamos que mantenemos la aplicación original abierta desde Visual Studio en el puerto 5091 y luego intentamos iniciar la DLL publicada en ese mismo puerto. Si ambos procesos solicitan una combinación incompatible de dirección y puerto, el segundo no podrá iniciar normalmente su escucha.

¿Qué debemos hacer? Identificar qué proceso está atendiendo la dirección, detener el que ya no necesitamos o elegir una dirección diferente para el ensayo. **No debemos modificar HomeController, reinstalar EF Core ni cambiar el nombre de las vistas**, porque el problema ocurre antes de que una solicitud alcance esas capas.

Cuando el objetivo es aprender, es útil provocar deliberadamente una colisión y observar el error. Pero al terminar debemos dejar solo un proceso utilizando el puerto; de otro modo, la siguiente práctica podría consultar sin darse cuenta una instancia anterior y atribuirle resultados al código recién publicado.

5.3.12. La petición HTTP como prueba y como límite de una afirmación

Examinemos este comando:

```
curl.exe -i http://localhost:5091/Productos/Estado
```

La opción `-i` solicita mostrar las cabeceras junto al cuerpo. Si recibimos `HTTP/1.1 200 OK` y el texto esperado, podemos afirmar que el servidor ha atendido correctamente **esa ruta** en ese momento.

No podemos afirmar sin más que el CRUD funciona o que la base SQL Server está disponible. Esa segunda afirmación requeriría ejecutar una acción real de lectura o modificación que use EF Core y comprobar su resultado. Las pruebas tienen alcance: deben corresponder a lo que decimos haber demostrado.

El código 503 también es una respuesta HTTP válida. En el modo sin LocalDB, el middleware de MVC A la genera para rutas que dependen de SQL. Esto demuestra que el proceso atiende la conexión, identifica la limitación y la expresa. No es la misma situación que un navegador incapaz de conectar porque no hay servidor escuchando.

5.3.13. Ejemplo razonado de un diagnóstico de tres capas

El alumno abre `http://localhost:5091/` y recibe una página HTML. Después consulta `/Productos` y recibe 503. Finalmente consulta `/css/site.css` y obtiene 200. ¿Dónde está el fallo?

No es correcto afirmar que Kestrel está caído: ha respondido en las tres solicitudes. Tampoco podemos concluir que falta la hoja de estilos. La evidencia apunta a una restricción aplicada al conjunto de rutas dependientes de SQL Server, algo coherente con el modo `Despliegue:SinLocalDb=true`.

En otro caso, el navegador no logra conectar con ninguna ruta y la consola del proceso muestra un error de dirección ocupada. Aquí la causa se sitúa **antes** del enrutamiento. De nuevo, investigar `ProductosController` sería improductivo.

En un tercer caso, `/` devuelve 200, `/Productos/Estado` devuelve 404 y `/css/site.css` devuelve 200. El servidor parece operativo, pero el endpoint concreto no está disponible. Debemos revisar el código que realmente se publicó y sus rutas, así como asegurarnos de no estar hablando con un proceso antiguo que permanece escuchando.

5.3.14. Qué significa detener correctamente la demostración

Al terminar una práctica, detener el proceso no es una formalidad. Permite liberar la dirección de escucha, evitar el consumo de recursos y garantizar que el siguiente resultado proceda del nuevo modo de hospedaje y no de la aplicación anterior.

En una terminal que ejecuta `dotnet`, Ctrl+C normalmente inicia la terminación. Después conviene comprobar que la ventana o el proceso ya no sigue anunciando el puerto y que una solicitud no recibe la misma respuesta. Si todavía responde, puede existir otro proceso distinto usando la dirección.

El alumno debe poder describir tres evidencias: el proceso de publicación se inició desde los artefactos, una ruta concreta respondió por HTTP y el proceso dejó de atender al detenerlo. Ese recorrido de inicio, operación y parada es la base para comprender el hospedaje posterior en IIS y el ciclo de vida del contenedor Docker.

5.3.15. El recorrido de una petición dentro del host MVC

La publicación de MVC A utiliza middleware y rutas de ASP.NET Core para transformar solicitudes en respuestas. Este recorrido suele parecer invisible cuando el alumno abre simplemente una página, por lo que conviene describirlo en pasos.

El programa configura el servicio de archivos estáticos, el enrutamiento y los endpoints MVC/Razor Pages. Una versión reducida de sus instrucciones finales es:

```
app.UseStaticFiles();
app.UseRouting();

app.MapRazorPages();
app.MapControllerRoute(
    name: "default",
```

```
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

El fragmento presupone que `app` ya se ha construido y que MVC/Razor Pages fueron registrados en los servicios. Es decir, explica el final de `Program.cs`, no una aplicación completa que el alumno deba copiar en otra solución.

Cuando llega `GET /`, ASP.NET Core puede seleccionar por convención el controlador `Home` y la acción `Index`, que produce una vista. Cuando llega `GET /css/site.css`, el middleware de archivos estáticos puede resolver el recurso dentro de `wwwroot` sin ejecutar una acción de listado de productos. Cuando llega `GET /Productos/Estado`, se invoca la acción que devuelve un texto, sin consultar EF Core.

Esta diferencia permite diagnosticar qué tramo del pipeline está implicado. El servidor HTTP es común; la producción concreta de la respuesta no tiene por qué serlo.

5.3.16. La respuesta HTTP no es solo el contenido que dibuja el navegador

Cuando utilizamos el navegador vemos sobre todo una página o un mensaje. Sin embargo, la respuesta HTTP posee estado, cabeceras y, normalmente, un cuerpo. La herramienta `curl.exe -i` permite observar una parte más explícita de esa comunicación.

Para la ruta de estado de MVC A, podemos esperar una respuesta con código satisfactorio y un cuerpo de texto semejante a:

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8

Catalogo MVC disponible
```

Las cabeceras exactas pueden incluir otros valores y variar con el entorno. Lo importante aquí es distinguir la línea de estado del contenido. El cuerpo permite verificar que ha respondido la acción adecuada. El estado 200 indica que esa solicitud terminó satisfactoriamente.

Un recurso inexistente puede producir 404; una ruta de productos deshabilitada para el modo sin LocalDB responde 503. En ambos casos el navegador puede mostrar una página de error, pero no estamos ante la misma situación técnica.

5.3.17. El significado de una respuesta 503 deliberada

La aplicación MVC A contiene una opción llamada `Despliegue:SinLocalDb`. Se ha añadido para demostrar Kestrel, IIS y Docker sin afirmar que SQL Server está disponible bajo cualquier identidad o sistema operativo.

Cuando esa opción es verdadera, el host omite la inicialización LocalDB y un middleware intercepta solicitudes a las rutas de productos que requerirían la base, con una excepción para `/Productos/Estado`. El middleware produce 503 y un mensaje de diagnóstico.

Podemos resumir el criterio:

```
¿La aplicación está ejecutándose con SinLocalDb=true?
    |
    +-- No --> recorrido normal de MVC con SQL Server Express LocalDB
    |
```

```
    +-- Sí --> ¿es una ruta de productos dependiente de SQL?
                   |
                   +-- Sí --> respuesta HTTP 503
                   |
                   +-- No --> continúa el pipeline de MVC y recursos
```

Una solicitud que recibe 503 ha llegado a un servidor capaz de atender HTTP. Por eso no debe describirse del mismo modo que una conexión rechazada. Tampoco se ha creado un catálogo alternativo en memoria: se ha expresado una capacidad que no está disponible en ese escenario.

La distinción importa cuando se comparan hospedajes. No sería legítimo afirmar que el CRUD se ha desplegado sobre Linux solo porque el inicio y la hoja CSS funcionen. En cambio, sí podemos afirmar que el host MVC se ha iniciado y responde correctamente las rutas independientes de SQL.

5.3.18. Un proceso antiguo puede producir una prueba engañosa

Imaginemos que el alumno ejecuta el proyecto con `dotnet run` en 5091, publica una versión nueva y olvida detener el proceso anterior. Después intenta iniciar la DLL publicada en ese mismo puerto, observa un error de dirección ocupada y vuelve al navegador. La portada sigue respondiendo, por lo que concluye erróneamente que la publicación está funcionando.

En realidad el navegador puede estar consultando el proceso antiguo que seguía escuchando. La respuesta HTTP confirma la disponibilidad de **algún proceso** en esa dirección, no su identidad ni la carpeta que utiliza.

Para resolver esta ambigüedad debemos comprobar el proceso de escucha y la salida de la terminal correspondiente. En Windows, `Get-NetTCPConnection` puede revelar el identificador del proceso asociado al puerto. La publicación nueva debe iniciarse tras detener la antigua, o en otro puerto claramente identificado.

Este ejemplo explica por qué los estados finales de las prácticas incluyen detener procesos y liberar puertos. No es burocracia: protege la trazabilidad entre una acción concreta y el resultado que se observa.

5.3.19. Los puertos forman parte del entorno, no de la identidad del proyecto

El mismo ensamblado MVC puede escuchar en 5091 o 5094 si se inicia con opciones compatibles y no existen conflictos. Eso no crea dos aplicaciones distintas desde el punto de vista del código; crea dos instancias de ejecución.

Si ambas utilizan el modo sin LocalDB, pueden atender las rutas sin base con independencia entre sí. El controlador Home/Index no necesita duplicarse para responder en dos puertos. La dirección de escucha se decide durante el alojamiento del proceso, no en la vista Razor.

Esta idea conecta el ensayo Kestrel con Docker: dentro del contenedor MVC escucha en 8080, mientras Docker puede publicar ese puerto en 8081 o 8082 del host. Lo que cambia es el acceso al proceso, no su lógica de negocio. El alumno debe aprender a mantener separadas las URL de Blazor, de la API auxiliar y de MVC para no atribuir resultados a un servidor equivocado.

5.3.20. El error de arranque precede al error de endpoint

Cuando el proceso intenta iniciar Kestrel, pueden fallar condiciones que nada tienen que ver con un controlador específico. Un puerto ocupado, un runtime ausente o una excepción durante la inicialización LocalDB pueden impedir que el servidor llegue a escuchar.

Por otro lado, cuando Kestrel escucha y recibe una solicitud, pueden aparecer problemas posteriores: una ruta inexistente, un recurso estático ausente o una acción que devuelve 503. Estos errores pertenecen a una fase distinta del recorrido.

Criterio de diagnóstico:

- Si no existe dirección de escucha y ningún recurso responde, investigar primero el arranque.
- Si el servidor responde 404 en una ruta concreta, investigar la ruta y los endpoints.
- Si la portada responde 200 y Productos 503 bajo modo sin base, interpretar la restricción declarada.
- Si la página HTML carga pero CSS responde 404, investigar los recursos publicados y la ruta estática.

La secuencia impide convertir todos los problemas en «Kestrel está roto». Un diagnóstico útil necesita identificar el punto en que el comportamiento deja de coincidir con el contrato.

5.3.21. La salida de consola es una observación, no una prueba de todas las funciones

Los mensajes del host muestran información de arranque y posibles excepciones. Ayudan a saber qué direcciones está escuchando Kestrel. Pero no sustituyen una solicitud de navegador o de un cliente HTTP a la ruta que deseamos comprobar.

Un mensaje de «Listening» indica que el proceso ha podido abrir la escucha; no acredita que exista `/Productos/Estado`. Una respuesta 200 a esa ruta demuestra un recorrido concreto, pero no que el CRUD pueda modificar una base SQL. Un formulario visible sin enviarlo no demuestra que `ModelState` y EF Core persistan correctamente.

Esta jerarquía de evidencias es útil para todas las modalidades de hospedaje. Los logs de Docker permiten ver el proceso del contenedor; la consola IIS permite ver un sitio configurado; solo una petición HTTP demuestra qué respuesta recibe realmente un cliente.

5.3.22. Separar el comportamiento del proceso y la aplicación al detenerlo

Durante una sesión de clase, suele haber varias terminales abiertas. Algunas ejecutan Blazor, otras la API, otras MVC. Detener la terminal equivocada puede provocar un error que parezca surgir de la última modificación del código. Por eso cada práctica establece qué proceso necesita y cuál puede detenerse.

En P5.3 la aplicación de referencia es solo MVC A. Se puede detener Blazor y la API sin afectar a las rutas de MVC que no consumen esos servicios. Cuando detenemos Kestrel con Ctrl+C, liberamos el puerto de esa instancia. Si la dirección sigue respondiendo, hay que averiguar si permanece otro proceso activo.

El aprendizaje consiste en asociar cada resultado observado con una instancia de ejecución concreta. Al pasar a IIS, el mecanismo de inicio cambiará; al pasar a Docker, la instancia recibirá un nombre de contenedor. La disciplina de identificar **quién está escuchando** seguirá siendo la misma.

5.3.23. Una ruta conocida como punto de partida para diagnosticar Kestrel

Para comprobar un servidor no necesitamos empezar con la parte más compleja de su aplicación. MVC A conserva una ruta `/Productos/Estado` que responde texto desde una acción del controlador. Este punto de observación no utiliza la tabla de productos. Su utilidad no consiste en crear un sistema general de monitorización, sino en aislar un recorrido pequeño y verificable.

Si obtenemos respuesta 200 en Estado, sabemos que algún proceso escuchó la dirección, que ASP.NET Core atendió la solicitud y que la acción pudo devolver su contenido. Si a la vez `/Productos` devuelve 503 bajo el modo sin base, no hay contradicción: dos rutas distintas recorren condiciones distintas.

Si Estado devuelve 404 mientras `/` funciona, el servidor sigue siendo capaz de atender solicitudes. Debemos comprobar si estamos ejecutando la publicación correcta, si la ruta pertenece a esa versión o si otra aplicación se encuentra escuchando el puerto. Un cambio arbitrario de puerto no es suficiente para diagnosticar un fallo de enrutamiento.

Ejemplo de razonamiento completo: al abrir `localhost:5091` el navegador muestra una portada correcta. Sin embargo, la URL de Estado no existe. El primer dato no autoriza a concluir que todo el proyecto MVC esté cargado con las rutas esperadas. Comprueba la versión publicada y si el proceso realmente ejecuta el proyecto acumulativo. Es posible que la dirección la esté atendiendo una plantilla MVC diferente que el alumno dejó iniciada.

5.3.24. El valor de comprobar el cuerpo además del código de estado

Una respuesta HTTP con código 200 no identifica por sí sola cuál de dos aplicaciones está contestando. Dos servidores distintos pueden devolver 200 en `/`. Por eso un endpoint de texto conocido resulta útil como firma funcional del proyecto que pretendíamos iniciar.

Al consultar `/Productos/Estado` esperamos `Catalogo MVC disponible`. La combinación de estado y cuerpo ofrece más información que un «OK» aislado. No demuestra identidad criptográfica ni sustituye mecanismos de seguridad, pero en un laboratorio permite descartar que estemos viendo simplemente la portada predeterminada de otro proyecto.

La misma técnica se utiliza al comparar las dos publicaciones de la práctica anterior. Ambas pueden responder 200 en Home/Index, pero una incorpora una marca específica en el HTML. El cuerpo de respuesta identifica la versión a la que accedemos.

Esto enseña una regla de observación general: cuando el objetivo consiste en demostrar **qué aplicación o versión se ejecutó**, necesitamos comparar algo más que la existencia de una respuesta satisfactoria.

5.3.25. La ruta de archivos estáticos no ejecuta el mismo trabajo que una acción MVC

La ruta `/css/site.css` pertenece a un archivo físico publicado bajo `wwwroot`. La petición puede ser atendida por el middleware de archivos estáticos sin invocar una acción del controlador de productos. En cambio, `/Productos/Estado` se resuelve a una acción MVC y su respuesta la produce código del controlador.

El servidor HTTP es común, pero las responsabilidades no son idénticas. Si falla CSS, revisaremos la publicación del archivo y el middleware. Si falla Estado, revisaremos la ruta de controlador y la versión de la aplicación. Si falla el catálogo, además podríamos necesitar comprobar SQL Server Express LocalDB.

En términos de prueba, solicitar tres tipos de recursos aporta tres observaciones:

Tipo de recurso	Ejemplo	Capacidad observada
Vista MVC	`/`	Enrutamiento y representación HTML
Acción de texto MVC	`/Productos/Estado`	Acción de controlador accesible
Archivo estático	`/css/site.css`	Publicación y servicio de recursos

La tabla no significa que todas las aplicaciones deban incorporar un endpoint de estado. Este proyecto dispone de uno y lo aprovechamos para explicar el hospedaje con un recorrido mínimo.

5.3.26. El mismo programa puede escucharse en más de una dirección

Supongamos que ejecutamos la publicación en 5091. Para iniciar una segunda instancia con la misma DLL debemos elegir otra dirección compatible, como 5094, si el primer proceso continúa escuchando. Ambas instancias tienen su propio ciclo de vida y su propia configuración de proceso.

En el modo sin LocalDB, la portada y la ruta de estado pueden responder de forma equivalente en los dos puertos. No hemos duplicado el código fuente, ni diseñado un segundo controlador; solo hemos ejecutado dos procesos. Esa diferencia se observa al detener una de las terminales: la otra puede seguir atendiendo solicitudes.

En el modo normal, los dos procesos podrían necesitar acceso a LocalDB y sus resultados dependerían de la disponibilidad de la instancia. Por eso un experimento destinado a comprobar exclusivamente una colisión o independencia de puertos debe aislar explícitamente esa dependencia y no adjudicar una excepción SQL a Kestrel.

5.3.27. Qué demuestra una colisión y por qué reproducirla puede ser útil

Una colisión de puertos ocurre cuando dos procesos intentan atender combinaciones incompatibles de dirección y puerto. Kestrel no puede iniciar su escucha si la combinación ya está ocupada de manera incompatible.

La observación es pedagógicamente relevante porque se produce **antes** de que el navegador ejecute una petición a Home/Index. No intervienen el model binding, las vistas Razor ni el contexto EF Core en la decisión del sistema operativo de admitir un listener.

Podemos describir el ensayo: iniciamos una publicación en 5091 y confirmamos que Estado responde. Sin detenerla, iniciamos otra publicación en 5091. El segundo proceso debe informar del conflicto, siempre que haya llegado a la fase de escucha. La primera instancia continúa respondiendo.

La corrección más pequeña es detener el proceso que ya no se necesita o utilizar otro puerto. No es necesario cambiar el código de controladores, reinstalar el SDK o suprimir la persistencia de la aplicación como reacción improvisada.

5.3.28. Los puertos no explican por sí solos la identidad del proceso

Aun cuando sepamos que 5091 está ocupado, necesitamos averiguar **por quién**. El sistema operativo puede mostrar el identificador del proceso que escucha. En Windows, PowerShell permite realizar esa consulta:

```
Get-NetTCPConnection -LocalPort 5091 -State Listen -ErrorAction SilentlyContinue |
    Select-Object LocalAddress, LocalPort, OwningProcess
```

Si aparece un identificador de proceso, puede consultarse `Get-Process` para obtener más información. Debe actuarse con prudencia: no se debe terminar un proceso solo porque usa el puerto; puede pertenecer a otro servicio del equipo o a otra persona.

En una máquina de formación suele bastar con detener la terminal concreta desde la que arrancó la aplicación. El comando de observación sirve para resolver casos en los que existen varias terminales y se ha perdido la correspondencia entre proceso y aplicación.

La presencia de un listener tampoco demuestra que cualquier endpoint HTTP funcione. Necesitamos la petición a una ruta concreta para comprobar el recorrido de aplicación.

5.3.29. La dirección efectiva es más importante que la recordada por el alumno

El comando de arranque puede contener `--urls http://localhost:5091`. Sin embargo, si la aplicación incorpora otra configuración específica de endpoints, la dirección efectiva podría diferir. No debemos enseñar que todos los mecanismos de configuración tienen la misma prioridad ni que una opción textual prevalece incondicionalmente en cualquier host.

En el proyecto de laboratorio, la práctica utiliza una configuración sencilla y se espera que la dirección anunciada coincida con la prevista. Cuando exista discrepancia, hay que leer el mensaje de escucha del proceso, comprobar sus fuentes de configuración y solicitar la URL efectiva.

La respuesta adecuada no consiste en añadir instrucciones a `HomeController`. Los controladores se ejecutan después de que ASP.NET Core haya recibido una solicitud a través de su mecanismo de hospedaje; la selección de puerto es una responsabilidad anterior.

5.3.30. El tratamiento de un 404 frente a una conexión rechazada

Si un navegador recibe HTTP 404, hubo comunicación con un servidor que atendió y rechazó la ruta o recurso. Si informa de que no puede conectarse, puede que no exista ningún listener accesible en esa dirección. Estos resultados no deben recibir la misma interpretación.

Por ejemplo, pedir `http://localhost:5091/ruta-inventada` a un proceso MVC correctamente iniciado puede producir 404. La prueba demuestra que Kestrel está funcionando y que el recurso solicitado no está definido. En cambio, detener el proceso y pedir Estado puede producir un error de conexión, que no contiene una respuesta 404 de MVC.

Este contraste resulta particularmente importante después de copiar una publicación. Un 404 puede indicar que el proyecto ejecutado no contiene la ruta prevista; un error de conexión puede indicar que ni siquiera se ha iniciado. Antes de diagnosticar EF Core debemos separar los dos casos.

5.3.31. Una publicación Kestrel no es exclusiva de desarrollo

El hecho de que usemos `dotnet run` durante la construcción no convierte a Kestrel en un servidor que solo pueda utilizarse en desarrollo. Una publicación ASP.NET Core puede atender solicitudes directamente mediante Kestrel cuando el entorno está configurado adecuadamente.

En este curso ejecutamos la DLL publicada en un puerto local. No se configuran certificados públicos, balanceadores ni un proxy externo. Esa limitación de alcance no debe confundirse con una propiedad intrínseca de Kestrel.

La diferencia que se enseña es entre **ejecutar desde la estructura de proyecto** y **ejecutar los artefactos publicados**. Ambos recorridos pueden utilizar el mismo servidor HTTP, aunque sus herramientas, carpetas y condiciones de arranque difieran.

Al introducir IIS veremos que el mecanismo de hospedaje Windows añade un módulo y un Application Pool. Al introducir Docker veremos que la misma aplicación puede arrancar en un contenedor. Ninguna de esas modalidades convierte los controladores MVC en otro tipo de programa.

5.3.32. La respuesta 503 es información de aplicación, no de conectividad

En el modo `Despliegue:SinLocalDb=true`, el programa reconoce rutas de productos que dependerían de SQL Server y devuelve una respuesta 503. La solicitud llegó al proceso; el servidor pudo identificar la condición y escribir el estado de respuesta.

Si el alumno observa HTTP 503 y afirma que el servidor está apagado, contradice la evidencia: un servidor ha producido precisamente esa respuesta. El mensaje expresa indisponibilidad de una función bajo la configuración elegida. No significa que toda la red esté rota.

En cambio, si la aplicación no consiguió iniciar porque intentó abrir LocalDB al arrancar y no pudo, quizá no habrá ningún listener disponible para responder con HTTP. Las dos situaciones tienen causas y síntomas distintos; ambas dependen de la configuración de base, pero aparecen en fases diferentes.

5.3.33. Elegir la siguiente comprobación a partir de la evidencia disponible

El alumno ve una página de error. ¿Qué hace a continuación? Depende de lo que ya se sabe.

Si no se recibió respuesta HTTP y la consola no anuncia una dirección de escucha, investigar el arranque y el puerto. Si hubo HTTP 404, comprobar la URL y la tabla de rutas. Si hubo HTTP 503 para productos y el modo sin base está activo, interpretar la restricción, no intentar repararla como si fuera inesperada. Si hubo HTML 200 sin CSS, inspeccionar los recursos publicados.

La técnica es un árbol de decisión conceptual:

```
¿Llega una respuesta HTTP?
  NO -> proceso, puerto, escucha, dependencia de inicio
  SÍ -> ¿qué ruta y qué código?
          200 -> comprobar también contenido y recurso esperado
          404 -> ruta, versión, archivo o endpoint ausente
          503 -> comprobar la dependencia o limitación declarada
          otro -> examinar la evidencia concreta antes de intervenir
```

El árbol no sustituye herramientas de diagnóstico; orienta su uso. Evita la modificación de archivos al azar cuando todavía no se ha determinado la capa del problema.

5.3.34. Cierre: reproducir el mismo resultado en el siguiente hospedaje

La comprobación de portada, acción de estado y CSS nos proporciona una referencia para comparar con IIS y Docker. Si la misma MVC A responde en otro mecanismo, la continuidad se

observa mediante las mismas rutas. Si una de ellas falla, conocemos qué parte del comportamiento está ausente.

No es necesario crear una prueba de cada controlador de M02 dentro de esta jornada. El CRUD completo ya tuvo su recorrido de aprendizaje con SQL Server LocalDB. M05 enseña un problema distinto: cómo iniciar, exponer y detener una publicación, y cómo identificar sus límites.

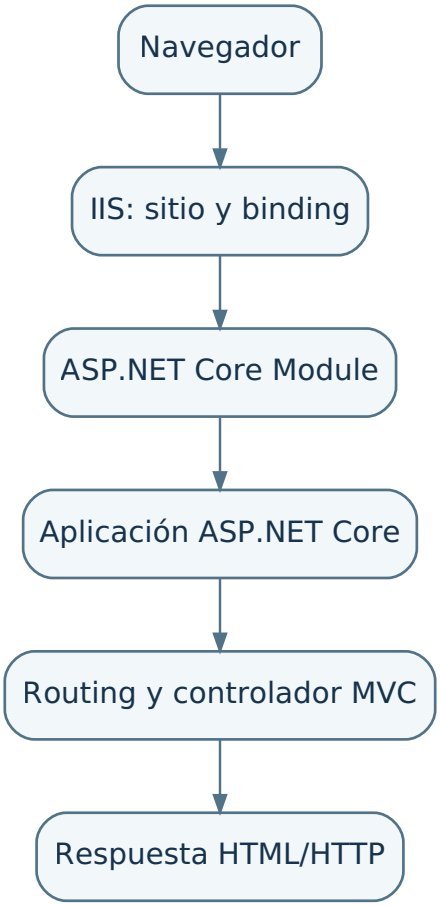
Este cierre conecta el apartado con la siguiente práctica de IIS: mantendremos constantes las rutas y cambiaremos el modo de hospedaje. Así el alumno compara mecanismos sin confundirlos con una modificación funcional de la aplicación.

5.4. Publicación elemental de ASP.NET Core en IIS

5.4.1. El problema que IIS resuelve

En Windows, IIS ofrece mecanismos para administrar sitios, enlaces de direcciones, procesos trabajadores y configuración de hospedaje. No reemplaza el código MVC ni convierte los controladores en páginas estáticas. En un hospedaje ASP.NET Core habitual, IIS trabaja con el **ASP.NET Core Module (ANCM)** y una aplicación publicada.

El esquema se puede simplificar así:



El modelo de hospedaje concreto puede ser **in-process** u **out-of-process**. En el primero, la aplicación ASP.NET Core se hospeda dentro del proceso trabajador IIS; en el segundo, IIS reenvía solicitudes a un proceso ASP.NET Core que utiliza Kestrel. Es importante no afirmar que *siempre* existe el mismo proceso Kestrel externo cuando utilizamos IIS. Para esta jornada basta con reconocer que ANCM conecta el mundo IIS con ASP.NET Core.

5.4.2. Hosting Bundle y ASP.NET Core Module

Instalar IIS en Windows no garantiza que conozca cómo iniciar una aplicación ASP.NET Core .NET 8. El **ASP.NET Core Hosting Bundle** instala o proporciona los componentes de hospedaje requeridos, incluido el módulo ANCM y los runtimes correspondientes. Si el bundle se instaló antes de habilitar IIS o el servicio no detecta el módulo, puede ser necesario reparar la instalación y reiniciar los servicios según las instrucciones del producto.

Un error habitual es confundir el SDK para desarrollar con el runtime/Hosting Bundle del servidor. El servidor IIS del laboratorio necesita componentes de hospedaje, no necesariamente Visual Studio. Tampoco basta con copiar una carpeta de código fuente al directorio del sitio.

5.4.3. Sitio, ruta física y binding

Un sitio IIS asocia una carpeta física de contenido publicado con uno o varios bindings. En el laboratorio utilizaremos HTTP local en un puerto concreto; por ejemplo, `localhost:8091`. En un escenario empresarial se requeriría tratar HTTPS y exposición pública, pero no forma parte de esta práctica introductoria.

El `Application Pool` proporciona un contexto de proceso e identidad. Para aplicaciones ASP.NET Core se configura habitualmente con **No Managed Code** respecto al CLR de .NET Framework; ello no significa que la aplicación carezca de código administrado .NET 8. Significa que el runtime moderno lo gestiona el mecanismo de hospedaje ASP.NET Core, no la configuración tradicional del CLR de .NET Framework del pool.

Los permisos del directorio publicado son otra condición de ejecución. Una identidad que no puede leer la carpeta o sus archivos impedirá iniciar o servir correctamente la aplicación. No concederemos permisos globales a todo el equipo como solución improvisada; delimitaremos los permisos necesarios para la identidad del sitio de ensayo.

5.4.4. La trampa de LocalDB bajo una identidad distinta

El CRUD MVC A fue desarrollado desde Visual Studio bajo la sesión interactiva del alumno. **SQL Server Express LocalDB está asociado a un usuario de Windows**, por lo que el hecho de disponer de una instancia desde Visual Studio no garantiza que el proceso de IIS, ejecutado bajo una identidad de Application Pool, pueda acceder a esa misma base.

Tenemos dos objetivos distintos:

1. Comprobar el mecanismo elemental de publicación y hospedaje IIS, para lo cual bastan rutas MVC sin base de datos.
2. Comprobar CRUD y persistencia LocalDB, lo cual exige permisos, usuario y configuración de la instancia adecuados; esa preparación excede la comparación mínima de hospedajes de M05.

Por eso el laboratorio utiliza rutas sin dependencias SQL para la prueba elemental de IIS. **No reemplazamos la base de datos ni eliminamos el CRUD**. En Windows/Kestrel puede utilizarse

la configuración normal para demostrar la persistencia; en IIS, la práctica se concentra en mostrar la aplicación y navegar rutas independientes de LocalDB.

5.4.5. Reconocer los fallos típicos

Síntoma	Qué comprobar primero	Qué no debemos concluir
IIS devuelve una página de error de hospedaje	Hosting Bundle, ANCM y logs	Que el controlador Index contiene un error
El sitio responde en un puerto diferente	Binding y conflicto con otro sitio	Que publicar cambia automáticamente el puerto
Error de acceso a los archivos	Carpeta física e identidad del pool	Que hay que dar control total a todos los usuarios
Página inicial funciona, CRUD falla	LocalDB, identidad y conexión	Que la publicación MVC está completamente rota
IIS muestra una versión antigua	Carpeta física y artefactos realmente publicados	Que basta actualizar el código fuente de Visual Studio

En un entorno de validación sin Windows/IIS es posible comprobar la estructura de la publicación y el archivo `web.config`, pero **no afirmar que el despliegue IIS funciona**. Una prueba de Kestrel no sustituye esta evidencia.

5.4.6. Una comparación explicativa

La aplicación MVC construida desde M01 no cambia sus controladores al desplegarse. Si la acción Home/Index devolvía una vista durante `dotnet run`, seguirá devolviéndola cuando reciba una petición válida bajo IIS, siempre que la configuración de hospedaje sea correcta. Esa independencia es precisamente una ventaja del framework: el código HTTP de ASP.NET Core no se reescribe para cada mecanismo elemental de hospedaje.

Lo que sí cambia es cómo se inicia el proceso, cómo llega la petición y qué identidad y recursos tiene el host. Por ello, una comparación correcta evalúa esas condiciones, manteniendo estable la aplicación que recibe las solicitudes.

5.4.7. Síntesis

El alumno debe ser capaz de publicar a carpeta, identificar los componentes IIS necesarios, crear un sitio con binding local, comprobar una ruta y distinguir un error de IIS/ANCM de uno causado por una dependencia como LocalDB.

5.4.8. Un sitio IIS no es una aplicación compilada ni una base de datos

Cuando hablamos de «publicar en IIS» estamos conectando tres objetos con responsabilidades diferentes: un directorio que contiene una publicación ASP.NET Core, un sitio de IIS que recibe solicitudes y un mecanismo de hospedaje capaz de iniciar la aplicación. Una base de datos SQL Server, si hace falta, constituye una dependencia adicional y no está incluida implícitamente en ninguno de esos tres objetos.

La carpeta publicada se genera con `dotnet publish`. IIS no compila el código C# de la solución durante una petición. El sitio asocia una ruta física con un enlace de escucha. El ASP.NET Core Module participa en la puesta en marcha y el tratamiento del tráfico hacia la aplicación.

Es posible que un sitio figure como «Started» en el administrador IIS y que la aplicación ASP.NET Core no consiga iniciarse. El estado administrativo del sitio no equivale a una respuesta HTTP correcta de Home/Index. Para afirmarlo necesitamos abrir la URL que corresponde al binding y comprobar el resultado real.

También puede suceder lo contrario: una publicación arranca perfectamente como proceso Kestrel independiente, pero falla bajo IIS. Eso no demuestra que la aplicación MVC contenga un error de lógica. El hospedaje IIS añade su módulo, identidad de proceso, configuración y permisos, todos ellos relevantes para el diagnóstico.

5.4.9. La relación entre sitio, binding, pool y carpeta física

El **sitio** es una unidad de configuración administrada por IIS. El **binding** indica en qué combinación de protocolo, dirección, puerto y, cuando corresponde, nombre de host acepta solicitudes. La **ruta física** indica dónde se encuentran los archivos publicados. El **Application Pool** define el contexto de ejecución y las condiciones bajo las que se procesan las solicitudes.

En el laboratorio elegimos `http://localhost:8091/`. El número 8091 no surge de ASP.NET Core ni de un controlador: es el puerto que decidimos vincular al sitio. Para solicitar esa dirección, IIS debe tener un binding compatible y el navegador debe conectar al host correcto.

La misma DLL podría copiarse a otra carpeta, pero si el sitio sigue apuntando a la carpeta anterior continuará utilizando los artefactos anteriores. Este es el origen de una situación frecuente: «he publicado la versión nueva y sigo viendo la antigua». No hay que reescribir el controlador para resolverlo; se debe verificar la carpeta física que está utilizando el sitio.

Respecto al Application Pool, **No Managed Code** significa que no seleccionamos un CLR tradicional de .NET Framework para ejecutar la aplicación ASP.NET Core. No significa «aplicación sin .NET», ni autoriza a omitir el Hosting Bundle. El sistema moderno de hospedaje ASP.NET Core requiere sus propios componentes.

5.4.10. Por qué el Hosting Bundle importa y cómo reconocer su ausencia

Una instalación de IIS preparada únicamente para páginas o aplicaciones tradicionales no dispone necesariamente del módulo requerido para hospedar ASP.NET Core 8. El Hosting Bundle incorpora los componentes necesarios para que IIS pueda iniciar una aplicación publicada para ese entorno, incluido ANCM.

La instalación de Visual Studio en el equipo de desarrollo no prueba que el servidor IIS de otro equipo tenga ese bundle. Tampoco basta con copiar una DLL o `web.config`. La máquina que recibe las solicitudes debe estar preparada de forma explícita, y la versión de runtime disponible debe ser compatible con la aplicación que se pretende ejecutar.

Cuando se instala IIS y después el Hosting Bundle, puede ser necesario reiniciar los servicios para que detecten el módulo. Si se instaló el bundle antes de habilitar IIS, el procedimiento de preparación del entorno puede requerir reparar su instalación. Lo importante para el alumno no es memorizar cada pantalla del instalador, sino entender **qué capacidad proporciona cada pieza**.

Si el módulo de hospedaje falta, el problema aparece antes de que los controladores MVC puedan atender normalmente sus rutas. Por eso una página de error de arranque bajo IIS debe llevarnos primero a revisar el hosting, la versión de runtime y los registros de inicio, no directamente a cambiar `Views/Home/Index.cshtml`.

5.4.11. Dos modelos de hospedaje, una misma responsabilidad MVC

En ASP.NET Core con IIS distinguimos los modelos *in-process* y *out-of-process*. En el primero, la aplicación se hospeda en el proceso trabajador IIS mediante el mecanismo correspondiente. En el segundo, IIS actúa delante de un proceso ASP.NET Core separado que atiende solicitudes mediante Kestrel.

Esta distinción es suficiente para interpretar diagramas y errores. No es necesario construir dos laboratorios distintos con sus configuraciones de rendimiento; eso convertiría una práctica elemental en un curso de administración de IIS.

Lo que debe permanecer estable en ambos conceptos es el papel de nuestra aplicación MVC. Los controladores siguen transformando solicitudes en resultados; las vistas siguen produciendo HTML; `wwwroot` sigue proporcionando recursos estáticos. El mecanismo de alojamiento cambia cómo se inicia y conecta ese código con el servidor externo.

Afirmación a evitar: «En IIS todo el tráfico debe pasar siempre por un proceso Kestrel externo». Solo describe de forma aproximada el hospedaje fuera de proceso y no el modelo in-process. La teoría debe permitir reconocer esa diferencia sin convertirla en un problema que el alumno tenga que configurar durante los 270 minutos de prácticas de M05.

5.4.12. Una lectura conceptual de `web.config`

En una publicación ASP.NET Core orientada a IIS, podemos encontrar instrucciones de hospedaje como las siguientes, presentadas aquí de forma ilustrativa:

```
<configuration>
  <system.webServer>
    <handlers>
      <add name="aspNetCore"
           path="*"
           verb="*"
           modules="AspNetCoreModuleV2"
           resourceType="Unspecified" />
    </handlers>
    <aspNetCore processPath="dotnet"
                arguments=".\CursoWeb.Mvc.dll"
                hostingModel="inprocess" />
  </system.webServer>
</configuration>
```

El archivo concreto lo genera la publicación según las propiedades del proyecto. No debemos sustituirlo por esta transcripción sin comprobar su contexto: aquí estudiamos **la función de sus elementos**, no una edición manual obligatoria. `AspNetCoreModuleV2` identifica el módulo. El ensamblado indicado representa la aplicación publicada. El modo de hospedaje informa cómo se ejecuta con IIS.

El alumno puede utilizar esta lectura para comprender por qué un `web.config` que apunta a otro ensamblado, una ruta física incorrecta o un módulo no instalado impedirían arrancar la aplicación. Pero la corrección no consiste en modificar a ciegas el XML: la salida generada y las condiciones reales del servidor son la referencia inicial.

5.4.13. Identidad del proceso y acceso a recursos externos

El Application Pool utiliza una identidad concreta. Esa identidad necesita acceso de lectura a los archivos publicados. Si el sitio no puede leer su carpeta, los fallos se sitúan en permisos de sistema de archivos o configuración de IIS, no en la sintaxis del controlador MVC.

SQL Server Express LocalDB introduce una segunda dimensión. Una instancia utilizada por el desarrollador al ejecutar Visual Studio suele pertenecer a su sesión o contexto de usuario. El proceso IIS puede utilizar otra identidad y, por tanto, **no compartir automáticamente la posibilidad de acceder a esa instancia**. Una cadena de conexión idéntica en `appsettings.json` no elimina esa diferencia.

El recorrido principal del curso separa estos problemas: verificamos el hospedaje IIS con páginas y rutas sin SQL, y dejamos el CRUD con LocalDB para el contexto Windows en que fue preparado y probado. No se instala otra base, no se asignan permisos globales a todos los usuarios como solución indiscriminada y no se finge que los endpoints de productos funcionan sin infraestructura.

Este criterio no empobrece la demostración. La hace precisa: un alumno puede afirmar «he servido por IIS una aplicación ASP.NET Core real» si ha solicitado sus rutas y recursos y ha observado la respuesta. No puede afirmar «he desplegado el CRUD persistente» si no ha comprobado la base, sus permisos y operaciones.

5.4.14. Diagnóstico de una página inicial que no aparece

Imaginemos que IIS tiene un sitio configurado y el navegador devuelve un error en lugar del HTML esperado. La estrategia de diagnóstico puede ordenarse así:

Primero, confirmar la URL, el puerto y el binding. Una solicitud a otro puerto podría estar llegando a un sitio distinto o no llegar a ninguno.

Segundo, confirmar que la ruta física apunta **a la publicación** y no a la carpeta de código fuente o a una publicación antigua. La existencia de `CursoWeb.Mvc.dll` y los demás artefactos es una condición que debemos comprobar.

Tercero, revisar que IIS conoce el ASP.NET Core Module y que está instalado el runtime requerido. Si la aplicación ni siquiera se inicia, estudiar primero sus dependencias de hospedaje.

Cuarto, verificar los permisos mínimos de acceso a la carpeta. Una identidad sin permiso de lectura puede impedir el funcionamiento aunque todos los archivos estén presentes.

Quinto, si la aplicación ha iniciado pero una ruta particular falla, analizar su código y dependencias. Una página inicial que responde no demuestra que las rutas CRUD puedan comunicarse con LocalDB.

Por último, comprobar que se está visitando la versión recién publicada. Si la aplicación parece antigua, comparar el binding, la ruta física y la fecha de los artefactos. Esta comprobación es mejor que volver a compilar repetidamente sin saber qué publicación está ejecutando IIS.

5.4.15. Caso explicado: IIS sirve la portada, pero no el catálogo

Un alumno termina el ensayo y presenta las siguientes observaciones:

Recurso	Resultado observado
`/`	HTTP 200, página MVC

Recurso	Resultado observado
`/css/site.css`	HTTP 200, hoja de estilos
`/Productos/Estado`	HTTP 200, texto esperado
`/Productos`	HTTP 503

La interpretación es precisa: IIS recibe solicitudes, ANCM hospeda la aplicación, el enrutamiento y los recursos estáticos funcionan, y la aplicación rechaza intencionadamente las rutas de datos en el modo sin LocalDB.

No corresponde «arreglar» esa última respuesta habilitando una base inventada ni editando la acción de listado para devolver siempre una colección fija. Eso haría parecer que el CRUD funciona y borraría precisamente la diferencia que se está enseñando. Para demostrar el CRUD de M02 deberíamos elegir un entorno con SQL Server Express LocalDB real y probar operaciones reales bajo una identidad autorizada.

Esta disciplina de separar hechos comprobados y capacidades no comprobadas es transferible a cualquier tecnología de hospedaje. Una portada disponible es una evidencia limitada; una operación de base funcional sería una evidencia distinta.

5.4.16. El sitio IIS y el programa ASP.NET Core tienen ciclos de vida diferentes

Al abrir el Administrador de IIS, el alumno ve sitios y grupos de aplicaciones. Son objetos de administración. La existencia de un sitio no implica que se haya publicado una DLL ASP.NET Core válida; tampoco que su proceso haya llegado a iniciar.

Un sitio define dónde y cómo recibe solicitudes IIS. El directorio físico contiene los artefactos publicados. El Application Pool establece un contexto de ejecución, y el ASP.NET Core Module enlaza ese mundo con el inicio de nuestra aplicación. Si falta cualquiera de las piezas necesarias, la experiencia del navegador puede ser diferente.

Una página de error antes de que Home/Index pueda ejecutarse no debe interpretarse automáticamente como un fallo de la vista. Primero se comprueban los componentes de hospedaje, el archivo `web.config`, el runtime y los permisos. Cuando una ruta específica falla después de que la portada ya funcione, el diagnóstico se desplaza hacia esa ruta y sus dependencias.

La separación del ciclo de vida es importante porque el docente podría encontrar un sitio `Started` con una aplicación que no responde. «Started» describe el estado de configuración administrado por IIS, no un contrato de respuesta HTTP satisfactoria para todas las rutas.

5.4.17. Elegir un binding no modifica el código de HomeController

En el laboratorio creamos un sitio `CursoWeb-M05` asociado al puerto 8091. La URL `http://localhost:8091/` permite solicitar su portada desde el mismo equipo. El número pertenece a la configuración de entrada de IIS, no a una propiedad del controlador MVC.

Podemos imaginar que el mismo contenido publicado se hospeda en otro puerto libre. La acción `HomeController.Index` seguiría recibiendo solicitudes a la ruta prevista. El cambio de dirección no exige duplicar el controlador ni añadir condiciones del tipo «si el puerto es 8091».

Esta independencia explica una ventaja central del framework: la lógica de aplicación se construye alrededor de solicitudes, rutas, acciones y respuestas. El mecanismo que acepta inicialmente la conexión puede variar entre Kestrel directo, IIS y contenedor Docker.

No obstante, no debemos confundir la independencia del controlador con ausencia de configuración. Los bindings, puertos, certificados y nombres de host forman parte del entorno. Un binding incorrecto puede impedir que una solicitud alcance siquiera el endpoint MVC.

5.4.18. La ruta física debe apuntar a una publicación, no a la carpeta de la solución

Un error frecuente consiste en seleccionar como ruta física de IIS la carpeta donde vive el `.csproj`. Allí existen fuentes y configuraciones de desarrollo, pero no necesariamente una publicación adecuada. El mecanismo de hospedaje IIS espera los artefactos preparados con `dotnet publish`.

La práctica utiliza `C:\CursoWeb\M05\IIS` como destino y publica directamente a esa carpeta. Después el sitio IIS debe apuntar a ese mismo directorio. Si el alumno ejecuta otra publicación en `.work\M05` y olvida cambiar el destino, IIS seguirá sirviendo la versión que se encuentre en la carpeta que tiene configurada.

Es útil comparar dos preguntas: «¿a dónde escribió los archivos `dotnet publish`?» y «¿qué carpeta física utiliza el sitio IIS?». Solo cuando ambas respuestas corresponden al mismo destino podremos atribuir el contenido del navegador a esa publicación.

Este razonamiento es idéntico al ejercicio de dos publicaciones Kestrel, pero aquí interviene una configuración administrativa adicional. La aplicación sigue siendo MVC A; la diferencia es quién selecciona sus artefactos y cómo inicia el proceso.

5.4.19. Por qué la identidad del pool cambia el significado de una misma ruta SQL

En Visual Studio, el proyecto MVC se ejecuta bajo la identidad del alumno. SQL Server Express LocalDB suele estar asociado a esa identidad. En IIS, el Application Pool puede ejecutarse bajo otra cuenta; incluso si la cadena de conexión es idéntica, el acceso real a LocalDB puede diferir.

La confusión nace de atribuir demasiada capacidad al archivo `appsettings.json`. Una cadena de conexión es un dato de configuración. No crea por sí misma una sesión Windows, no traslada credenciales ni instala SQL Server. El proceso que utiliza la cadena debe tener acceso real al destino indicado.

En el recorrido de M05 no reconfiguramos el pool para darle acceso a una instancia LocalDB del usuario. Eso introduciría permisos y particularidades de administración que exceden el objetivo simple de publicación IIS. Por ello activamos el modo sin base para demostrar de manera independiente las rutas que no requieren SQL.

La consecuencia observable es que `/` y `/Productos/Estado` pueden responder 200, y `/Productos` debe devolver 503. El código 503 no es un defecto de IIS; es la manera en que la propia aplicación declara la indisponibilidad del CRUD bajo el escenario elegido.

5.4.20. El archivo de entorno puede estar presente y no ser el que el proceso utiliza

La práctica crea `appsettings.Production.json` en la carpeta publicada. En un host cuyo entorno efectivo sea Production, ese archivo aporta la clave `Despliegue:SinLocalDb=true`. Si el entorno se ha configurado con otro nombre, el hecho de tener ese archivo no demuestra que se haya cargado.

El alumno necesita conocer dos valores: el entorno con el que arrancó realmente ASP.NET Core y la ubicación del archivo que define la opción. Solo con esa información puede interpretar de manera fiable por qué el host omite o intenta utilizar LocalDB.

La corrección debe ser específica. Si el sitio ejecuta otro entorno, hay que proporcionar la configuración en la fuente que realmente consulta ese proceso. No es correcto crear múltiples archivos de entorno al azar hasta que alguno produzca una portada.

Este caso resulta especialmente instructivo al comparar Kestrel e IIS: durante Kestrel una variable se configura en la terminal del alumno; en IIS la aplicación se inicia bajo la administración del sitio y no hereda sin más cualquier valor escrito en una PowerShell independiente.

5.4.21. Qué significa comprobar ANCM y por qué no basta una casilla de IIS

ANCM, el ASP.NET Core Module, conecta el servidor IIS con el mecanismo de hospedaje ASP.NET Core. Es necesario porque IIS no adquiere automáticamente soporte de .NET 8 por haber habilitado una característica de servidor web de Windows.

El Hosting Bundle aporta los componentes para alojar aplicaciones ASP.NET Core, incluido el módulo y los runtimes necesarios. En el laboratorio lo instalamos antes de declarar que la publicación puede funcionar bajo IIS.

Un error de inicio puede derivarse de un módulo ausente, de un runtime incompatible o de un `web.config` que no corresponde al programa publicado. Por eso copiar solamente `CursoWeb.Mvc.dll` al sitio no constituye un despliegue suficiente.

Debemos conservar también una distinción conceptual: en modo in-process la aplicación se hospeda dentro del proceso trabajador IIS mediante el mecanismo correspondiente; en out-of-process se utiliza un proceso ASP.NET Core separado. No enseñamos a optimizar ambas modalidades, pero sí evitamos afirmar que siempre tiene que existir el mismo Kestrel externo.

5.4.22. Interpretar un error sin conceder permisos excesivos

Una aplicación necesita leer sus archivos publicados. La identidad del pool debe tener los permisos necesarios para ello. Un fallo por acceso denegado puede resolverse concediendo permisos apropiados sobre la carpeta del sitio, pero no justifica dar control total a todo el equipo.

En un laboratorio compartido, ampliar indiscriminadamente permisos crea un problema de seguridad que no es necesario para aprender IIS. La respuesta es identificar la identidad que ejecuta el pool, el directorio físico y la operación que ha fallado.

La misma precaución se aplica al acceso SQL: conceder permisos globales sobre una instancia de base de datos para hacer desaparecer un error de LocalDB no forma parte de esta práctica. El modo sin base evita tener que realizar esa preparación para demostrar la portada y los recursos estáticos.

Ejercicio conceptual: el sitio ha sido creado y IIS informa de un error de lectura de archivos. ¿Debemos cambiar `HomeController`? No: el fallo puede impedir que la aplicación llegue a

ejecutarse. La información disponible orienta a revisar el acceso a la carpeta física y la identidad del pool.

5.4.23. Una prueba IIS debe solicitar la URL IIS

Una prueba con `dotnet run` demuestra que la aplicación puede ejecutarse directamente. No demuestra que un sitio IIS haya aceptado una solicitud ni que ANCM esté instalado correctamente. La práctica utiliza la dirección de binding `http://localhost:8091/` para observar el mecanismo real.

El alumno debe comprobar al menos la portada MVC, la acción de estado y el CSS en ese puerto. Después puede consultar Productos para confirmar la restricción 503 del modo sin LocalDB. Las solicitudes se dirigen a IIS, no a una instancia Kestrel anterior que sigue escuchando en 5091.

También hay que verificar que el sitio utiliza la carpeta publicada correcta. Una página inicial con estado 200 podría proceder del sitio equivocado si el alumno ha abierto el puerto de otra aplicación. La relación entre nombre del sitio, binding, ruta física y contenido recibido es parte de la evidencia.

5.4.24. El proyecto fuente no cambia al dar de alta un sitio

Durante el apartado IIS no se modifican los controladores MVC. Los archivos que cambian pertenecen a la configuración del destino: copia publicada, archivo de entorno específico, sitio, binding y Application Pool.

Si para conseguir que la portada funcione alguien elimina `UseSqlServer`, no ha comprobado el despliegue de la misma aplicación. Ha cambiado su arquitectura funcional. El curso exige mantener el CRUD de M02 para el entorno Windows con LocalDB, y aislar únicamente las condiciones del ensayo de hospedaje.

Esta distinción será útil al pasar a Docker: tampoco allí crearemos otra aplicación MVC. Se generará una imagen a partir del mismo proyecto y se activará una configuración que indica la ausencia de LocalDB en Linux.

5.4.25. Comparación entre un sitio configurado y una ruta funcional

Observación	Conclusión que permite	Conclusión que no permite
El sitio existe en IIS	La configuración fue creada	Que ASP.NET Core haya arrancado
El pool figura Started	IIS gestiona ese pool	Que Home/Index devuelva HTML
La carpeta contiene la DLL	Hay un artefacto publicado	Que exista el runtime
GET `/` devuelve 200 desde 8091	La ruta de portada funciona a través del binding	Que LocalDB esté accesible
GET CSS devuelve 200 desde 8091	IIS y ASP.NET Core sirven el recurso	Que el CRUD funcione
GET Productos devuelve 503 esperado	El modo sin base está aplicado a esa ruta	Que la consulta SQL se haya realizado

La tabla evita un error recurrente: acumular comprobaciones de administración y declararlas equivalentes a una operación de aplicación. La publicación web debe describirse mediante solicitudes y respuestas, no solo mediante estados en una consola gráfica.

5.4.26. Cierre: IIS añade infraestructura sin volver a escribir MVC

Al finalizar, el alumno debería poder explicar cómo una publicación construida fuera de Visual Studio se coloca en una carpeta utilizada por IIS, cómo el sitio define una dirección de entrada, cómo el pool establece un contexto de proceso y cómo ANCM permite hospedar ASP.NET Core.

La misma acción Home/Index continúa produciendo HTML, pero bajo condiciones de arranque diferentes. LocalDB sigue siendo la dependencia funcional original y no se declara disponible sin haberlo comprobado.

El siguiente apartado conserva el proyecto MVC y cambia otra vez el entorno: ejecutará una imagen Docker Linux. La transición desde IIS no debe llevarnos a reescribir controladores; debe enseñarnos a identificar la nueva forma de empaquetado, los puertos y las limitaciones reales del proceso.

5.5. Docker: ejecutar la misma aplicación MVC en un contenedor Linux

5.5.1. Imagen, contenedor y aplicación no son lo mismo

Una **imagen** es una plantilla de capas con el sistema de archivos y la configuración necesarios para iniciar una aplicación. Un **contenedor** es una instancia de ejecución creada a partir de una imagen. Un Dockerfile describe cómo construir esa imagen; no es por sí mismo una aplicación ejecutándose.

No confundamos estos términos con una máquina virtual completa. El contenedor comparte el kernel del host mediante mecanismos de aislamiento, con las particularidades de Docker Desktop en Windows, que utiliza una infraestructura de virtualización/WSL para ejecutar contenedores Linux.

La aplicación MVC continúa siendo ASP.NET Core. Docker no sustituye el servidor HTTP: el proceso `dotnet CursoWeb.Mvc.dll` inicia ASP.NET Core y su servidor dentro del contenedor. Docker proporciona un entorno de ejecución empaquetado, configuración de red y control del ciclo de vida.

5.5.2. El papel de un Dockerfile multietapa

Para un proyecto ASP.NET Core .NET 8 resulta instructivo separar la etapa de compilación, que usa el SDK, de la etapa final de ejecución, que usa un runtime ASP.NET Core más pequeño:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS compilacion
WORKDIR /src
COPY . .
RUN dotnet restore CursoWeb.Mvc.csproj
RUN dotnet publish CursoWeb.Mvc.csproj -c Release --no-restore -o /app/publicada

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS ejecucion
WORKDIR /app
COPY --from=compilacion /app/publicada .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "CursoWeb.Mvc.dll"]
```

`FROM` determina una imagen base. `WORKDIR` fija el directorio de trabajo. `COPY` incorpora archivos. `RUN` ejecuta instrucciones **durante la construcción** de la imagen. `ENTRYPOINT` establece el proceso que arrancará **al ejecutar** un contenedor. Confundir ambas fases explica por qué no basta con añadir un `RUN dotnet publish` para servir una página web.

La etapa final no requiere el SDK completo. El ejemplo presupone que ejecutamos `docker build` desde la carpeta que contiene el `.csproj`, porque `COPY . .` depende del **contexto de construcción** que pasamos a Docker. Lanzar el comando desde una carpeta distinta sin adaptar rutas puede producir un error «proyecto no encontrado» aunque el Dockerfile sea correcto.

5.5.3. Puerto interno y puerto del host

En una imagen ASP.NET Core 8, el puerto de escucha que utilizaremos es **8080 dentro del contenedor**. La orden:

```
docker run --rm -p 8081:8080 cursoweb-mvc:m05
```

publica el puerto 8080 del contenedor en el puerto 8081 del host. Desde el navegador de Windows pedimos `http://localhost:8081/`. `EXPOSE 8080` documenta el puerto previsto, pero no realiza por sí mismo la publicación hacia el host. Para acceder desde la máquina utilizamos `-p`.

Al confundir los dos números, alguien puede pedir `http://localhost:8080/` y no ver la aplicación, aunque esté respondiendo correctamente en el puerto 8081 del host. El diagnóstico empieza por revisar el mapeo configurado, no por modificar el controlador MVC.

5.5.4. ¿Por qué no usamos LocalDB dentro del contenedor?

La aplicación A de M02 utiliza SQL Server Express LocalDB, instalado en Windows y ligado a una cuenta de usuario. **No existe de modo ordinario dentro de una imagen Linux ASP.NET Core**. Copiar `appsettings.json` no instala SQL Server ni traslada la instancia existente.

Sería técnicamente posible diseñar una arquitectura de contenedores y un SQL Server accesible externamente, con gestión de conexiones y persistencia. Pero ello introduciría conceptos ajenos al objetivo elemental de esta sesión, además de romper la simplificación del temario. Tampoco sería correcto cambiarla sin explicaciones a SQLite: alteraríamos el proveedor y la semántica del ejemplo.

El laboratorio opta por una separación explícita: en el ensayo Docker se activa `Despliegue:SinLocalDb=true`. Esto omite únicamente la inicialización de base del arranque y rechaza con 503 las rutas de productos que dependen de SQL Server. Las páginas y recursos estáticos independientes siguen sirviéndose. La aplicación original conserva su CRUD para Windows y se puede volver a ejecutar allí sin esa opción.

La diferencia entre **servicio HTTP disponible** y **funcionalidad CRUD disponible** debe hacerse observable y explicarse, no ocultarse.

5.5.5. Construcción, ejecución y observación

La secuencia mínima que da sentido al Dockerfile es:

```
docker build -t cursoweb-mvc:m05 .
docker images cursoweb-mvc
docker run --name cursoweb-m05 --rm `
```

```
  -e Despliegue__SinLocalDb=true `
  -p 8081:8080 cursoweb-mvc:m05
```

El comando `docker build` crea una imagen. `docker images` permite comprobar que existe. `docker run` inicia una instancia. La variable de entorno define el modo del ensayo sin alterar el repositorio. Desde otra terminal, `docker ps` informa de contenedores activos; `docker logs cursoweb-m05` permite observar la salida de la aplicación. El comando `docker stop cursoweb-m05` solicita detenerla; `--rm` ordena eliminar el contenedor una vez terminado.

Si el puerto del host está ocupado, modificar temporalmente el primer número de `-p` puede resolver el conflicto. Cambiar el puerto interno sin modificar el servidor que escucha en 8080 no lo hará.

5.5.6. Ciclo de vida y ausencia de persistencia añadida

Los archivos que genera una aplicación dentro del contenedor no deberían considerarse persistentes por defecto. Cuando se elimina el contenedor, su sistema de archivos de escritura puede desaparecer. El curso no introduce volúmenes ni bases externas en este experimento: basta con entender por qué el resultado es adecuado para demostrar ejecución, pero no para almacenar el catálogo MVC.

Los contenedores también tienen logs y estado de salida. Que `docker build` tenga éxito no implica que `docker run` mantenga el proceso activo. Para investigar si el proceso termina inmediatamente, leeríamos `docker logs`, comprobaríamos que la DLL existe y revisaríamos la configuración. Un navegador mostrando error de conexión puede indicar que no hay un contenedor ejecutándose; no demuestra por sí solo que la imagen se construyó mal.

5.5.7. Comparación final de Kestrel, IIS y Docker

Dimensión	Kestrel en Windows	IIS en Windows	Docker Linux
Aplicación docente	MVC A	La misma MVC A	La misma MVC A
Artefactos	Salida `dotnet publish`	Carpeta publicada	Imagen multietapa
Inicio	Proceso `dotnet`	IIS + ASP.NET Core Module	`docker run` inicia `dotnet`
Puerto de ejemplo	5091	8091	Host 8081 → contenedor 8080
LocalDB	Disponible si la instancia está preparada	Limitada por identidad del pool	No disponible de forma ordinaria
Rutas verificadas mínimas	Inicio y estado sin base	Inicio y estado sin base	Inicio y estado sin base
CRUD completo	Solo tras comprobar LocalDB real	No forma parte del ensayo IIS elemental	No forma parte del ensayo Linux

La intención no es declarar ganadora a una modalidad, sino reconocer que **la misma aplicación puede publicarse de diferentes formas mientras cambian sus condiciones de ejecución**. La capa MVC sigue recibiendo solicitudes HTTP, pero varían los procesos, los permisos y los mecanismos de exposición del puerto.

5.5.8. La diferencia entre construir una imagen y ejecutar un proceso

Durante `docker build`, Docker interpreta el Dockerfile y produce una imagen. Los comandos `RUN` de su etapa de compilación se ejecutan en el proceso de construcción y preparan los archivos de aplicación. Al terminar, el hecho de tener una imagen no significa que exista un servidor web atendiendo solicitudes.

`docker run` crea y pone en marcha un contenedor a partir de esa imagen. En nuestro ejemplo, `ENTRYPOINT ["dotnet", "CursoWeb.Mvc.dll"]` define el programa que se ejecuta dentro del contenedor. Ese proceso inicia ASP.NET Core y su servidor HTTP. Si termina, el contenedor deja de estar en ejecución normal. Una imagen correcta puede, por tanto, producir un contenedor que termina inmediatamente debido a una mala configuración de ejecución.

La distinción puede expresarse con tres preguntas:

- ¿Existe una imagen construida con las capas previstas? Lo comprobamos con `docker images`.
- ¿Existe un contenedor en ejecución a partir de esa imagen? Lo comprobamos con `docker ps`.
- ¿El proceso del contenedor atiende una petición HTTP desde el host? Lo comprobamos abriendo la URL publicada mediante `-p`.

Estas observaciones no son sinónimos. Docker puede informar de un contenedor en marcha aunque la aplicación todavía esté inicializándose; también puede permitir crear un contenedor cuya ejecución termina antes de que nadie abra el navegador.

5.5.9. Por qué el contexto de construcción tiene tanta importancia como el Dockerfile

El comando `docker build -t cursoweb-mvc:m05 .` termina con un punto. Ese punto identifica el **contexto de construcción**. Las instrucciones `COPY` operan sobre los archivos enviados en ese contexto, no sobre cualquier ruta disponible en el equipo.

En nuestro Dockerfile aparece:

```
WORKDIR /src
COPY . .
RUN dotnet restore CursoWeb.Mvc.csproj
RUN dotnet publish CursoWeb.Mvc.csproj -c Release --no-restore -o /app/publicada
```

Para que `CursoWeb.Mvc.csproj` aparezca en `/src`, debemos usar como contexto la carpeta del proyecto MVC que lo contiene. Si el alumno invoca `docker build` desde la raíz del repositorio con ese mismo Dockerfile y sin ajustar el contexto, puede copiar una estructura de carpetas diferente. El error de restauración posterior no significa que el SDK .NET 8 esté roto; significa que el archivo no está en la ruta esperada dentro de la etapa de construcción.

`.dockerignore` permite excluir archivos que no necesitamos enviar al contexto. Un directorio `bin` o `obj` creado durante el desarrollo no debería presentarse como fuente del nuevo build. La etapa de SDK se encarga de restaurar y publicar a partir de los archivos correctos.

El ejemplo mantiene un Dockerfile multietapa sencillo. En proyectos grandes podríamos reorganizar el orden de copias para aprovechar más capas en caché, pero esa optimización no es necesaria para comprender el recorrido fundamental de esta jornada.

5.5.10. La imagen final no necesita las herramientas utilizadas para construirla

La primera etapa del ejemplo utiliza `mcr.microsoft.com/dotnet/sdk:8.0`. Contiene el SDK y herramientas de construcción. La etapa final utiliza `mcr.microsoft.com/dotnet/aspnet:8.0`, que proporciona el runtime ASP.NET Core requerido para ejecutar la publicación.

La instrucción `COPY --from=compilacion /app/publicada .` incorpora a la etapa final la salida preparada en la etapa anterior. El resultado no necesita transportar el código fuente completo ni el SDK para iniciar el servidor HTTP. Esta separación hace más evidente que «compilar» y «ejecutar» son responsabilidades diferentes, incluso cuando se describen en un mismo Dockerfile.

Sin embargo, una imagen final pequeña no elimina requisitos de la aplicación. Si un programa necesita un servicio SQL, seguirá necesitándolo. La etapa runtime de ASP.NET Core no incorpora mágicamente una instancia LocalDB. El ejercicio deja intencionadamente ese CRUD fuera del recorrido Linux para mantener la fidelidad técnica con SQL Server Express LocalDB del curso.

5.5.11. Comprender el mapeo de puertos sin cambiar el código MVC

Supongamos un contenedor cuyo proceso ASP.NET Core escucha en 8080. Cuando ejecutamos:

```
docker run --name cursoweb-m05 --rm -p 8081:8080 `
  -e Despliegue__SinLocalDb=true cursoweb-mvc:m05
```

Docker publica el puerto 8080 del contenedor a través del puerto 8081 del host. El navegador en Windows consulta `http://localhost:8081/`. El primer número de `-p` corresponde al lado del host, el segundo al del contenedor.

Si 8081 ya está ocupado, podemos elegir 8082 como puerto del host sin reescribir el Dockerfile, siempre que el proceso del contenedor continúe escuchando en 8080. Cambiar solamente el primer número resuelve la colisión exterior. Cambiar solamente el segundo a un puerto en el que la aplicación no escucha no proporciona acceso a la web.

La declaración `EXPOSE 8080` comunica la intención de usar ese puerto, pero **no crea automáticamente una publicación accesible desde el host**. Para la prueba elegimos explícitamente el mapeo con `-p`. Se trata de una decisión del entorno de ejecución y no de una ruta de controlador MVC.

5.5.12. Tres direcciones que el alumno debe mantener separadas

En la práctica aparecen al menos tres conceptos de dirección. La primera es la dirección que el servidor escucha **dentro del contenedor**; el Dockerfile la establece mediante `ASPNETCORE_URLS=http://+:8080`. La segunda es el puerto que Docker publica **en el host**, como 8081. La tercera es la URL concreta que abre el navegador del alumno, `http://localhost:8081/`.

No hay contradicción en que dos números distintos aparezcan en la misma aplicación. El navegador no tiene que conocer el puerto interno si utiliza el publicado por Docker. El servidor ASP.NET Core tampoco necesita saber qué número de host ha elegido el docente, porque sigue escuchando internamente en 8080.

Una avería típica consiste en abrir `http://localhost:8080/` desde el host y concluir que el contenedor no funciona. Si no se ha publicado ese puerto exterior, la solicitud no corresponde al mapeo real. El diagnóstico empieza por `docker ps`, donde podemos comprobar el nombre del contenedor, su estado y los puertos publicados.

5.5.13. Variables de entorno como decisiones de ejecución

Docker permite suministrar variables al iniciar el contenedor:

```
docker run --name cursoweb-m05 --rm `
  -e Despliegue__SinLocalDb=true `
  -p 8081:8080 cursoweb-mvc:m05
```

La variable hace que la aplicación .NET interprete la clave `Despliegue:SinLocalDb` como verdadera. El mismo ensamblado y la misma imagen podrían iniciarse con otros valores, pero sus dependencias reales cambiarían. No debemos ocultarlo detrás de la frase «Docker adapta automáticamente la base».

Con esta configuración la portada y los recursos sin dependencia SQL siguen disponibles. Las rutas del catálogo MVC que requieren LocalDB deben responder 503. El servidor está operativo; la funcionalidad CRUD no lo está. La variable permite demostrar un límite controlado sin modificar el proveedor EF Core ni inventar persistencia.

En un entorno productivo, la configuración requeriría criterios de seguridad y gestión de secretos que aquí no se desarrollan. La variable utilizada no contiene credenciales: expresa únicamente una opción didáctica de ejecución.

5.5.14. La vida de un contenedor y la vida de los datos no son equivalentes

Una imagen se conserva después de detener un contenedor, salvo que se elimine expresamente. Un contenedor creado desde esa imagen tiene un ciclo de vida de ejecución. Sus archivos de escritura no deberían utilizarse como almacenamiento persistente implícito de datos de negocio cuando el contenedor se elimina.

En el ejemplo usamos `--rm` para que Docker retire el contenedor al terminar. Esa elección simplifica la limpieza del laboratorio, pero significa que no esperamos conservar el estado de escritura de ese contenedor. No se incorporan volúmenes ni un SQL Server externo; sería necesario enseñar una infraestructura distinta para explicar correctamente esa persistencia.

Tampoco la presencia de una variable que evita LocalDB crea una base en memoria que admita el CRUD MVC. Al contrario: el middleware explica la indisponibilidad de las rutas que requerirían SQL. El resultado permite aprender el ciclo de vida de una aplicación real sin atribuirle capacidades que no tiene.

5.5.15. Leer `docker ps` y `docker logs` como pruebas con distinto alcance

`docker ps` informa de contenedores en ejecución y su configuración general. Nos ayuda a comprobar si el proceso sigue activo y cómo se han publicado los puertos. Si el contenedor no aparece, quizá terminó, no llegó a arrancar o fue eliminado tras su finalización por `--rm`.

`docker logs cursoweb-m05` muestra la salida registrada del contenedor mientras existe. Puede revelar errores de inicio de ASP.NET Core, direcciones de escucha y excepciones de aplicación. Es una evidencia diferente a la del navegador.

`curl.exe -i http://localhost:8081/Productos/Estado` pregunta algo más concreto: «¿qué responde esta ruta desde el host ahora mismo?». Aunque el contenedor figure en ejecución y los logs no contengan errores, debemos solicitar HTTP para demostrar que el mapeo de puertos y la ruta funcionan.

Por tanto, las tres comprobaciones se complementan: contenedor activo, salida del proceso y respuesta de red. Si una falla, conviene localizar en qué etapa se produjo la interrupción antes de reconstruir la imagen.

5.5.16. Caso razonado: la imagen se construye y el navegador no conecta

El alumno ejecuta `docker build` y obtiene una imagen. Después intenta abrir `http://localhost:8081/`, pero el navegador informa de que no se puede establecer conexión. ¿Podemos afirmar que el Dockerfile está mal? No todavía.

Primero debemos saber si se ejecutó `docker run`. La construcción no inicia automáticamente el servidor. Segundo, comprobar si el contenedor sigue en ejecución; un proceso que terminó puede haber dejado la imagen intacta. Tercero, examinar el mapeo de puertos efectivo. Si se publicó 8082:8080, la URL 8081 no corresponde al contenedor.

Solo después procede revisar los logs de arranque y el programa. Una excepción durante el inicio de ASP.NET Core, una variable de configuración ausente o una incompatibilidad en los artefactos publicados podrían explicar que el proceso no atendiera nunca una solicitud.

Conclusión del caso: debemos separar una imagen disponible, un contenedor activo y una aplicación web accesible. Comprobar una condición no garantiza automáticamente las restantes.

5.5.17. Caso razonado: HTTP 200 en la portada y 503 en productos

El mismo contenedor devuelve 200 en `/`, 200 en `/css/site.css` y 200 en `/Productos/Estado`, pero devuelve 503 en `/Productos`. Es un resultado esperado bajo el modo `Despliegue:SinLocalDb=true`.

La observación demuestra ejecución real del host, middleware de archivos estáticos y rutas MVC. La ruta de productos no contiene un catálogo ficticio para el contenedor: declara que la base SQL Server Express LocalDB no está disponible en ese entorno.

Sería improcedente sustituir LocalDB por SQLite solo para conseguir un 200 más. El objetivo pedagógico es comparar hospedajes y sus dependencias, no rediseñar la persistencia ni falsear una función. Si quisiéramos un CRUD Linux con SQL Server, habría que preparar de forma explícita otra infraestructura accesible al contenedor, dentro de un proyecto distinto al recorrido mínimo acordado.

5.5.18. Una comparación causal con los otros dos hospedajes

Con Kestrel en Windows iniciamos una publicación como un proceso `dotnet` y elegimos su dirección local. Con IIS en Windows registramos un sitio y el ASP.NET Core Module participa en su hospedaje. Con Docker creamos una imagen y ejecutamos un contenedor que inicia ASP.NET Core en un entorno Linux.

En todos ellos el código MVC conserva su responsabilidad de responder a solicitudes mediante controladores, vistas y archivos estáticos. Lo que cambia es el mecanismo de inicio, la identidad, el sistema de archivos visible, los puertos y las dependencias disponibles.

El objetivo no es decidir qué tecnología es mejor para todos los usos. Una elección empresarial exigiría evaluar mantenimiento, seguridad, redes, datos, observabilidad y operación, asuntos que esta jornada solo menciona para dejar claros sus límites. Aquí aprendemos a describir una publicación sencilla y a demostrar con observaciones verificables qué parte de la misma aplicación funciona bajo cada mecanismo.

5.5.20. Construir y ejecutar son verbos diferentes también dentro de Docker

El Dockerfile contiene instrucciones que se ejecutan en fases distintas. `RUN dotnet restore` y `RUN dotnet publish` pertenecen al proceso de construcción de la imagen. En cambio, `ENTRYPOINT ["dotnet", "CursoWeb.Mvc.dll"]` determina el comando que se ejecuta cuando se inicia un contenedor.

Un alumno puede ver que Docker ha restaurado dependencias y publicado la aplicación y concluir equivocadamente que el servidor debe estar activo. La construcción solo ha preparado capas y archivos; el proceso web nace cuando se ejecuta el contenedor. Por ello `docker images` y `docker ps` responden preguntas diferentes: la primera sobre imágenes disponibles, la segunda sobre instancias ejecutándose.

La separación coincide con la que ya aprendimos mediante `dotnet publish` y Kestrel. El contexto de ejecución se encapsula en una imagen, pero la existencia del artefacto no implica que un proceso atienda conexiones. La comprobación final continúa siendo una petición HTTP a una ruta conocida.

5.5.21. Seguir los archivos desde el contexto hasta la imagen de runtime

La primera etapa de nuestro Dockerfile recibe el proyecto MVC y ejecuta las operaciones de restauración y publicación. Para ello utilizamos como contexto de `docker build` la carpeta que contiene `CursoWeb.Mvc.csproj`.

La ruta de los archivos importa en dos sistemas de archivos: primero el directorio local que Docker recibe como contexto y después el directorio de trabajo de la etapa de compilación. `WORKDIR /src` establece una carpeta dentro de la imagen; `COPY . .` copia allí el material del contexto. `RUN dotnet restore CursoWeb.Mvc.csproj` presupone que el archivo existe ahora bajo `/src`.

La segunda etapa parte de una imagen ASP.NET Core de runtime y copia únicamente los artefactos publicados desde la etapa anterior. Los ensamblados se trasladan dentro del proceso de construcción; no se vuelven a compilar cuando el contenedor recibe cada solicitud HTTP.

Ejemplo de diagnóstico: Docker informa de que no existe `CursoWeb.Mvc.csproj` durante restore. El primer punto de inspección es el contexto que se pasó a `docker build` y la ruta de copia resultante. No es lógico comenzar por reinstalar SQL Server Express LocalDB, porque la restauración del proyecto todavía no ha iniciado el servidor web ni ha intentado consultar la base.

5.5.22. Por qué un `.dockerignore` forma parte del ejemplo

El proyecto contiene archivos de desarrollo que no necesitan enviarse al contexto de Docker: resultados locales de `bin` y `obj`, carpetas de Visual Studio y ciertos archivos de usuario. El `.dockerignore` impide que formen parte del contexto enviado a la construcción.

Esto no significa que los ensamblados finales vayan a faltar. El Dockerfile ejecuta `dotnet publish` dentro de su etapa SDK. La publicación adecuada se genera allí y luego se copia a la imagen runtime.

La distinción permite un experimento razonado: aunque el alumno borre las carpetas `bin` y `obj` locales de una copia limpia del proyecto, la imagen debería poder construirse si los archivos fuente y las dependencias necesarias están disponibles. La imagen no debe depender de haber compilado antes en Visual Studio.

En proyectos reales se prestarían otras atenciones al contexto, como impedir que se incluyan secretos. Este curso se limita a un `.dockerignore` elemental y a datos ficticios, pero el principio sigue siendo válido: una construcción necesita un contexto definido conscientemente.

5.5.23. Un contenedor Linux no contiene la sesión Windows del alumno

El entorno de ejecución de Docker Desktop para contenedores Linux no es el mismo contexto Windows en que Visual Studio utiliza LocalDB. Aunque el proyecto conserve la cadena de conexión de M02, no aparece automáticamente una instancia SQL Server Express LocalDB accesible dentro de la imagen.

Si arrancáramos la aplicación en el modo normal y su inicialización intentara abrir LocalDB, la ejecución podría fallar antes de atender HTTP. No deberíamos resolver ese fallo copiando cualquier DLL SQL ni declarando que el proveedor EF funciona sin comprobarlo.

El modo `Despliegue:SinLocalDb=true` responde a esta diferencia de entorno. No elimina `UseSqlServer` ni escribe otro controlador de productos; evita la inicialización incompatible con el objetivo del ensayo y produce un 503 para las rutas que necesitan datos.

Este límite está documentado en el código y en las prácticas para evitar que se confunda una aplicación MVC accesible con un CRUD persistente disponible. El ejercicio de Docker no incluye un servidor SQL externo ni un volumen, y no tiene que simularlos.

5.5.24. Escuchar en una interfaz interna no equivale a publicar un puerto del host

Dentro del contenedor, la aplicación necesita escuchar en una dirección compatible con el tráfico que Docker dirige a su puerto. Por eso el Dockerfile define `ASPNETCORE_URLS=http://+:8080`, de modo que ASP.NET Core acepta conexiones en el puerto interno previsto.

La opción `EXPOSE 8080` documenta la intención de la imagen. No proporciona por sí sola una dirección externa en Windows. Para que el navegador del host pueda acceder, ejecutamos `docker run -p 8081:8080`.

El orden de números importa. El puerto 8081 pertenece al lado del host y 8080 al contenedor. Si el alumno invierte la relación o solicita 8080 en Windows cuando publicó 8081, puede concluir erróneamente que la aplicación no responde. El diagnóstico correcto empieza por consultar el mapeo real que informa Docker.

5.5.25. Dos contenedores de la misma imagen no necesitan compartir puerto exterior

Una imagen puede utilizarse para crear más de un contenedor. Si cada uno escucha internamente en 8080, el host no necesita publicar ambos en el mismo puerto exterior. Podemos imaginar uno mapeado a 8081 y otro a 8082.

El programa MVC es idéntico en ambos; cada instancia tiene su proceso y su propio ciclo de vida. La necesidad de puertos exteriores distintos no implica cambiar el controlador Home/Index. Es una característica del entorno de alojamiento.

El laboratorio no requiere mantener dos contenedores simultáneos. Este razonamiento, no obstante, aclara por qué al resolver una colisión del puerto host se puede cambiar el primer número de `-p` conservando el segundo. Cambiar la escucha interna a otro valor exigiría coordinarla con el programa.

5.5.26. La variable de entorno pertenece al contenedor que se inicia

Cuando ejecutamos:

```
docker run -e Despliegue__SinLocalDb=true -p 8081:8080 cursoweb-mvc:m05
```

la opción `-e` aporta una variable al proceso del contenedor. No es una edición de `appsettings.json` del proyecto y tampoco cambia por sí misma una instancia Kestrel que continúe ejecutándose fuera de Docker.

La comparación con IIS es instructiva. Bajo IIS se creó un archivo de entorno en la carpeta publicada para el proceso hospedado por el sitio. Bajo Docker se entrega un valor cuando se lanza el contenedor. Los mecanismos son diferentes, pero ambos producen una opción que el mismo código ASP.NET Core sabe leer.

Si el alumno modifica la variable de su propia terminal Windows después de que el contenedor haya arrancado, no debe dar por hecho que el proceso interno cambiará automáticamente. Lo importante es la configuración efectiva suministrada al iniciar esa instancia. Para repetir el ensayo con otros parámetros es más claro detener el contenedor y crearlo con opciones explícitas.

5.5.27. Qué revela la secuencia imagen → contenedor → respuesta

Una comprobación de Docker necesita tres observaciones encadenadas. La primera confirma que se ha construido una imagen. La segunda confirma que un contenedor de esa imagen está activo. La tercera confirma que la aplicación sirve una ruta HTTP desde el host.

No podemos sustituir la tercera por las anteriores. Una imagen puede construirse correctamente y un contenedor puede arrancar, pero la URL solicitada puede utilizar un puerto diferente al publicado. También puede ocurrir que el proceso termine antes de aceptar solicitudes; una construcción correcta no garantiza un arranque correcto.

La forma de diagnóstico depende de la etapa en que falla la secuencia. Si no hay imagen, comprobar el contexto, Dockerfile y construcción. Si no hay contenedor activo, consultar la salida del proceso y los logs. Si hay contenedor y no hay HTTP, comprobar puertos, dirección de escucha y ruta.

5.5.28. Los logs deben relacionarse con la instancia que estamos comprobando

El comando `docker logs cursoweb-m05` permite examinar la salida del contenedor que tiene ese nombre. No muestra la salida de cualquier instancia que se haya ejecutado anteriormente con un nombre diferente.

Si se utiliza `--rm`, Docker elimina el contenedor después de que termine. Esta decisión es útil para limpiar el aula, pero significa que no podemos esperar conservar indefinidamente los logs de un contenedor ya retirado. Cuando investigamos un fallo de arranque, conviene observar la terminal que ejecutó `docker run` y consultar los logs mientras el contenedor todavía existe.

La misma precaución se aplicaba a Kestrel: una terminal antigua y otra nueva pueden mostrar mensajes diferentes. Para atribuir un resultado a la publicación correcta debemos identificar el proceso o contenedor concreto que atendió la solicitud.

5.5.29. El estado 503 bajo Docker no es una prueba de base de datos

Al solicitar `/Productos` en el contenedor Linux, el modo sin LocalDB devuelve intencionadamente 503. Esta respuesta demuestra que el middleware identificó una funcionalidad no disponible. No demuestra una consulta EF Core satisfactoria, ni que se haya instalado SQL Server dentro de la imagen.

En cambio, `/Productos/Estado` sí puede devolver 200, porque es una ruta independiente de la base. La portada y CSS también pueden responder satisfactoriamente. Juntas, estas respuestas acreditan que la aplicación MVC publicada funciona en el alcance de hospedaje previsto.

El alumno no debe elegir una interpretación extrema —«Docker no funciona» o «todo funciona»— sin precisar el recurso. Docker y la aplicación tienen capacidades observables diferentes. La práctica enseña a describirlas.

5.5.30. Tres preguntas que permiten interpretar la limpieza del laboratorio

Al ejecutar `docker stop`, solicitamos detener un contenedor. Si lo iniciamos con `--rm`, Docker debe retirarlo después de terminar. Sin embargo, la imagen utilizada puede continuar apareciendo con `docker images`. Eso no es una contradicción: imagen y contenedor son objetos distintos.

En la carpeta de proyecto, el Dockerfile sigue siendo un archivo de texto que describe cómo crear la imagen; tampoco desaparece al detener el contenedor. Y el código MVC original sigue existiendo con su configuración LocalDB para Windows.

Una limpieza correcta elimina o detiene exclusivamente los recursos de ensayo que ya no se necesitan. No debe borrar imágenes ajenas ni ejecutar comandos de limpieza masiva que afecten a otros proyectos del equipo.

5.5.31. Cierre comparado: el programa no cambia, su entorno sí

La misma aplicación MVC A puede ejecutarse directamente con Kestrel, alojarse en IIS o iniciarse dentro de una imagen Docker. Los controladores y las vistas conservan su responsabilidad funcional; las diferencias residen en artefactos, procesos, direcciones, identidades y disponibilidad de dependencias.

La base SQL Server Express LocalDB permanece como parte del proyecto del curso. No se sustituye a escondidas por otro motor. Cuando no existe un entorno apto para ese CRUD, el ejercicio se limita honestamente a páginas y recursos independientes de la base.

El resultado formativo final es una explicación técnica completa de la publicación: qué archivos se prepararon, quién los ejecutó, cómo llegó una petición HTTP y qué parte del comportamiento quedó demostrada. Ese criterio permanece válido aunque en otros cursos se estudien despliegues más avanzados.

5.5.32. Cuatro fases de un despliegue Docker correctamente descrito

Un despliegue elemental en Docker puede expresarse en cuatro fases: obtener el contexto de construcción, generar la imagen, ejecutar un contenedor y comprobar la aplicación mediante HTTP. Esta secuencia reúne lo aprendido en Kestrel y en publicación sin añadir infraestructura de operación empresarial.

En la primera fase debemos garantizar que el contexto contiene el proyecto MVC A y su Dockerfile. Si `COPY . .` copia una estructura distinta, la restauración o publicación no encontrará el `.csproj` esperado. La carpeta actual es una condición de la construcción.

En la segunda fase Docker utiliza la imagen SDK de .NET 8 para restaurar y publicar. Un build satisfactorio produce una imagen, pero todavía no significa que haya un servidor escuchando. El archivo `ENTRYPOINT` indica qué se ejecutará al crear una instancia.

En la tercera fase `docker run` inicia un contenedor de la imagen. El proceso ASP.NET Core debe arrancar con sus dependencias disponibles y escuchar en el puerto interno elegido. La variable `Despliegue__SinLocalDb=true` hace explícita la restricción del ensayo Linux.

En la cuarta fase el navegador de Windows solicita el puerto exterior que Docker ha publicado. La ruta `/Productos/Estado` permite comprobar el funcionamiento básico del host, y `/Productos` confirma la limitación 503 prevista.

El alumno debe saber indicar qué evidencia corresponde a cada fase: salida de build, presencia de imagen, contenedor activo y respuesta HTTP. No son equivalentes ni se implican automáticamente unas a otras.

5.5.33. Dos errores de contexto que parecen errores de programación

Un estudiante ejecuta `docker build -t cursoweb-mvc:m05 .` desde una carpeta que no contiene el proyecto MVC. La construcción informa de que no encuentra `CursoWeb.Mvc.csproj`. El error puede aparecer durante un comando `dotnet restore`, pero el problema real es que `COPY . .` ha copiado un contexto diferente al previsto. La solución consiste en situarse en la carpeta correcta y repetir la construcción, no en modificar la clase `AppDbContext`.

Otro estudiante utiliza el contexto correcto, pero ha alterado su `.dockerignore` de forma que excluye archivos necesarios. El resultado puede ser parecido: faltan archivos dentro de la etapa SDK. La investigación debe revisar qué se envía al contexto y qué entra en `/src`.

Estos ejemplos muestran que una orden de compilación puede fallar por razones externas al lenguaje C#. Antes de tocar controladores o servicios hay que localizar la fase en que falla la secuencia. Esta estrategia coincide con la seguida durante las publicaciones a carpeta en Windows.

5.5.34. El estado del contenedor y el resultado de una ruta pertenecen a planos distintos

`docker ps` permite saber si un contenedor está activo y observar cómo se han publicado sus puertos. Pero un contenedor activo no garantiza que una ruta MVC concreta devuelva 200: el programa podría servir 404 para una URL inexistente o 503 para una función deshabilitada.

El estado administrativo tampoco revela por sí solo que el navegador esté utilizando el puerto correcto. Si el contenedor escucha en 8080 y Docker ha publicado 8082:8080, una petición al host 8081 no corresponde al mapeo declarado. El contenedor puede estar perfectamente activo y responder correctamente a través de 8082.

Por eso la comprobación final siempre incluye la URL visible desde el host. Las herramientas de Docker explican el estado del contenedor; una solicitud HTTP verifica el recorrido hasta la aplicación.

5.5.35. La imagen no modifica la semántica de los controladores

El Dockerfile no introduce nuevos endpoints. La misma acción `HomeController.Index` genera una vista y la acción de Estado devuelve su texto de referencia. La etapa runtime ejecuta el ensamblado publicado de MVC A; no crea otro proyecto ni cambia la manera de seleccionar rutas.

Si la página inicial responde en Kestrel Windows y en Docker Linux, es una demostración de portabilidad del host y de esas funcionalidades concretas, siempre dentro de las dependencias disponibles. No se debe extender esa conclusión al CRUD con LocalDB sin pruebas adicionales y una infraestructura SQL accesible.

La presencia de una DLL con el mismo nombre tampoco garantiza que la imagen contenga la versión más reciente de la aplicación. Si se modifica una vista después de construir la imagen, el contenedor creado desde la imagen anterior continuará utilizando sus artefactos. Hace falta reconstruir una imagen e iniciar una instancia de esa nueva versión para observar el cambio.

5.5.36. La relación entre una imagen antigua y una pantalla antigua

Supongamos que el alumno modifica Home/Index en el proyecto fuente y abre de nuevo el navegador sobre el contenedor que ya estaba ejecutándose. La página no cambia. No hay que deducir que Razor está roto.

La imagen se construyó a partir de un estado anterior del proyecto. Su etapa de publicación preparó los ensamblados y vistas de ese estado. El contenedor que se ejecuta desde esa imagen no consulta continuamente los archivos fuente del equipo Windows.

El recorrido correcto consiste en guardar los cambios, volver a construir una imagen con un identificador controlado y ejecutar un contenedor de esa imagen. Después podremos comparar el contenido HTTP recibido. El principio ya se enseñó con las carpetas A y B de `dotnet publish`: los artefactos son estados concretos, no vínculos vivos con el editor.

Esta explicación conecta dos tecnologías aparentemente distintas mediante el mismo concepto de versión desplegada.

5.5.37. La limpieza forma parte de un experimento reproducible

Un laboratorio puede dejar imágenes, contenedores y procesos activos si el alumno solo cierra la pestaña del navegador. Esa situación dificulta la siguiente práctica: un puerto puede seguir ocupado o un nombre de contenedor puede existir aunque la aplicación ya no sea visible.

El comando `docker stop` solicita la detención de un contenedor que controlamos. Si fue creado con `--rm`, la retirada posterior forma parte de su configuración de ciclo de vida. La imagen puede permanecer disponible para iniciar un contenedor nuevo sin construirla de nuevo.

La limpieza responsable no consiste en eliminar todos los recursos Docker del equipo. El estudiante debe identificar por nombre el contenedor de ensayo, comprobar su estado y detenerlo sin afectar a otras aplicaciones. Esta precaución es análoga a no cerrar procesos Windows ajenos porque usan un puerto.

5.5.38. Las dependencias externas no desaparecen al utilizar contenedores

Docker encapsula un entorno de ejecución, pero no cambia mágicamente los contratos funcionales del programa. Si una aplicación necesita una base de datos, un servicio de archivos o una API externa, debe poder acceder a ellos desde el contenedor en las condiciones correspondientes.

MVC A utiliza SQL Server Express LocalDB en el recorrido Windows. El contenedor Linux del curso no incluye esa instancia ni configura una base externa. En consecuencia, el ensayo solo declara disponibles las rutas independientes de SQL y responde 503 para las dependientes.

Una versión empresarial podría configurar un SQL Server independiente y las credenciales y redes necesarias, pero eso exigiría más contenidos que los asignados a M05. No se sustituye de manera silenciosa LocalDB por SQLite para simular una funcionalidad completa.

El alumno aprende a reconocer este límite técnico y a decirlo expresamente. Esa precisión es una competencia más importante que conseguir una respuesta 200 mediante datos inventados.

5.5.39. Análisis final de una incidencia con resultados mixtos

Observamos lo siguiente: `docker build` termina correctamente, `docker ps` muestra un contenedor en marcha, `/` responde 200, `/css/site.css` responde 200 y `/Productos` responde 503. ¿Está mal el despliegue?

En el alcance acordado, el resultado es coherente. La imagen se construyó, el contenedor ejecuta el host, la ruta MVC de portada y el recurso estático responden y la ruta SQL está declarada no disponible. No hay evidencia de un error de publicación que deba corregirse.

Si por el contrario `/css/site.css` respondiera 404, deberíamos revisar el archivo `wwwroot` incluido en la imagen y la URL solicitada. Si todas las rutas fueran inaccesibles, investigaríamos el proceso y el mapeo de puertos. Si el contenedor terminara inmediatamente, leeríamos su salida para identificar un fallo de arranque.

Estos resultados orientan a capas diferentes. El alumno debe explicar la situación con el grado de precisión que permiten las observaciones, sin atribuir a Docker propiedades que corresponden al código ASP.NET Core o a la infraestructura SQL.

5.5.40. Lo que permanece después de finalizar los cinco módulos

El curso ha construido una aplicación MVC con Razor y SQL Server Express LocalDB y otra aplicación Blazor Interactive Server que consume una API docente. M05 no sustituye ninguna de ellas. Consolida la comunicación de B y demuestra distintas formas elementales de ejecutar A.

Desde el punto de vista didáctico, el cierre necesita integrar varias habilidades. El alumno debe comprender los archivos de un proyecto, interpretar el ciclo de vida de una llamada asincrónica, reconocer el papel de un servicio tipado, producir una publicación, identificar procesos y puertos y distinguir una funcionalidad web disponible de otra que exige una dependencia externa.

La finalidad de este recorrido no consiste en convertir seis horas de clase en un curso avanzado de administración, seguridad o contenedores. Consiste en conseguir que cada paso pueda explicarse, ejecutarse y comprobarse con el mismo proyecto que el alumno ha construido.

5.5.41. Caso integrador: tres respuestas distintas no significan que el curso tenga datos contradictorios

Al concluir el recorrido, imaginemos una demostración simultánea en el aula. Una persona abre el catálogo remoto de Blazor B y ve **Cuaderno y Lápiz**; otra abre el catálogo de MVC A desde Visual Studio y ve los productos almacenados en **SQL Server Express LocalDB**; una tercera consulta la ruta `/Productos` de la publicación MVC ejecutada en Docker y obtiene **503 Service Unavailable**. ¿Cuál de las tres observaciones es correcta?

Las tres pueden serlo. Para demostrarlo no debemos empezar modificando datos, sino identificando de dónde procede cada respuesta.

En **Blazor B**, el componente utiliza el servicio HTTP tipado para consultar la API auxiliar. Esa API mantiene productos en memoria y los devuelve mediante JSON. Una recarga de la página vuelve a pedir una representación a ese servidor; detener y reiniciar la API recrea sus valores iniciales. Nada de ello modifica automáticamente el catálogo de MVC A.

En **MVC A bajo Windows**, la acción que representa productos utiliza el contexto EF Core configurado con SQL Server Express LocalDB. Si la instancia existe y el usuario dispone de acceso, los productos pueden persistir entre ejecuciones del host. La interfaz HTML se genera mediante controladores y vistas Razor, sin utilizar el servicio de productos de Blazor B.

En **Docker Linux**, se ejecuta esa misma MVC A con `Despliegue:SinLocalDb=true`. El contenedor puede servir la portada y los recursos estáticos, pero la ruta CRUD recibe un 503 deliberado porque no se ha trasladado LocalDB al entorno Linux. La respuesta no ofrece una tercera colección de productos: declara que esa funcionalidad no participa en el ensayo.

Observación	Proceso responsable	Origen de datos o condición	Interpretación correcta
Blazor muestra dos productos	Blazor B consulta su API HTTP	Diccionario en memoria de la API	Estado remoto recibido por el componente
MVC muestra registros	Host MVC A en Windows	EF Core y SQL Server Express LocalDB	Vista generada a partir de consultas SQL
MVC Docker devuelve 503	Host MVC A dentro del contenedor	Modo explícito sin LocalDB	Hospedaje disponible, CRUD no comprobado

El caso pone de manifiesto una competencia transversal: **dos pantallas pueden compartir el nombre «Productos» sin compartir almacenamiento, proceso ni ciclo de vida**. Tampoco dos respuestas HTTP de una misma DLL acreditan las mismas dependencias.

Si el alumno comprende este contraste, estará en condiciones de explicar por qué debe consultar la API directamente al depurar Blazor, por qué necesita comprobar LocalDB al estudiar el CRUD MVC y por qué una prueba satisfactoria de IIS o Docker no sustituye una prueba real de persistencia.

La conclusión técnica no depende del número de comandos ejecutados: depende de poder reconstruir para cada resultado la secuencia **cliente → proceso → ruta → dependencia → respuesta**, sin confundir el contenido mostrado con la infraestructura que lo produjo.

5.5.42. Conclusión del curso

Tras M05 el alumno habrá recorrido dos ideas complementarias: una aplicación Blazor que se comunica correctamente con una API y una aplicación MVC que puede ser publicada y ejecutada en distintas modalidades de hospedaje. Habrá aprendido a distinguir compilación de ejecución, respuesta HTTP de funcionamiento completo, interfaz de servidor, y estado temporal de persistencia.

El criterio de éxito no es reproducir un comando aislado. Es poder explicar **qué proceso se está ejecutando, qué recurso sirve, qué dependencias utiliza y qué evidencia demuestra realmente que funciona**.